



N° d'ordre : XXX

**MEMOIRE DE FIN DE FORMATION EN VUE DE L'OBTENTION DU
DIPLOME DE LICENCE PROFESSIONNELLE**

Etablissement : Ipnet Institute of Technology (IIT)

Domaine : Sciences et Technologies

Mention : Informatique

Spécialité : Génie Logiciel

Promotion : 2023

THEME :

MISE EN PLACE D'UNE PLATEFORME WEB ET MOBILE DE
GESTION DES BIENS IMMOBILIERS RESIDENTIELS :
CAS DES COURS COMMUNES & AIRBNB

Présenté par :

GBANDI WAKE

Directeur de mémoire :

SHABAN Abdoulatif

Ingenieur Logiciel

DEDICACE

À mes parents pour leur soutien indéfectible tout au long de mon parcours académique,

À mon grand frère aîné, pour tous les sacrifices consentis tout au long de ce parcours académique.

À mes enseignants de l'IIT, dont les orientations ont façonné ma vision du génie logiciel.

À tous ceux qui croient que la technologie peut transformer le quotidien africain.

REMERCIEMENTS

Au terme de ce parcours académique, la réalisation de ce mémoire de fin de cycle a été rendue possible grâce au concours et au soutien de plusieurs personnes envers qui nous tenons à exprimer notre profonde gratitude.

- ✓ Au Tout-Puissant, pour sa grâce, sa protection continue et la force spirituelle qu'il nous a accordée tout au long de nos années d'études.
- ✓ À notre Directeur de mémoire, M. SHABAN Abdoulatif, Enseignant à IPNET & Ingénieur Logiciel, pour son encadrement technique rigoureux, sa disponibilité constante et ses conseils avisés qui ont guidé la réalisation de ce document et bien plus encore.
- ✓ À la Direction de Ipnnet Institute of Technology (IIT), pour avoir mis à notre disposition un cadre de formation moderne, propice à l'excellence et au développement de compétences de haut niveau.
- ✓ À l'ensemble de nos enseignants, pour la qualité de leur transmission pédagogique, leur rigueur et leur investissement quotidien qui ont forgé notre profil de jeune professionnel de l'informatique.
- ✓ À notre famille, pour leur soutien moral indéfectible, leurs prières et leurs sacrifices financiers continuels qui ont constitué le pilier de notre réussite.
- ✓ À nos camarades de promotion, pour l'esprit de solidarité et d'entraide constructive.

À toutes les personnes qui, d'une manière ou d'une autre, ont contribué à la concrétisation de ce travail, nous vous disons un sincère merci.

AVANT - PROPOS

Le présent mémoire s'inscrit dans le cadre de la fin du cycle de formation en vue de l'obtention du diplôme de Licence Professionnelle en Informatique, mention Génie Logiciel, à Ipnet Institute of Technology (IIT). Institution d'excellence reconnue pour former les professionnels de l'informatique de la prochaine génération, l'IIT met un accent particulier sur l'adéquation entre l'enseignement théorique et les réalités techniques du monde professionnel. À l'issue de ce parcours de trois ans, chaque étudiant est appelé à concevoir et réaliser une solution logicielle innovante afin de prouver sa maturité technique et son aptitude à intégrer le marché de l'emploi.

Ce document est le fruit d'un travail personnel et rigoureux, mené avec le souci constant de l'application des meilleures pratiques de l'ingénierie logicielle. De l'analyse des besoins au choix de l'architecture en couches, chaque étape a été pensée pour répondre à des critères stricts de performance, de sécurité et de maintenabilité.

L'idée de ce projet est née d'un constat terrain concernant les lacunes structurelles de la gestion locative résidentielle au Togo. À Lomé comme dans plusieurs localités, la gestion des biens immobiliers (qu'il s'agisse de la location classique en cours communes ou de la location de courte durée de type Airbnb) repose encore très largement sur des pratiques artisanales, manuelles et informelles. Le recours systématique aux registres papier, l'absence de traçabilité des contrats et la complexité du suivi des paiements entraînent régulièrement des pertes d'informations, des retards de recouvrement et des pertes de confiance entre bailleurs et locataires.

Face à ces défis, il nous est apparu indispensable de concevoir une plateforme numérique sur mesure, adaptée au contexte économique togolais (notamment par l'intégration des paiements par Mobile Money). Ce mémoire retrace ainsi l'ensemble de la démarche méthodologique et technique qui a guidé la mise en place de cette solution.

Malgré toute la rigueur accordée à ce travail, nous restons conscients qu'une œuvre humaine n'est jamais parfaite. C'est pourquoi nous accueillerons avec beaucoup d'intérêt et d'humilité les remarques, critiques et suggestions du jury visant à parfaire la qualité de ce projet.

RESUME

Ce mémoire présente la conception et le développement d'une plateforme numérique de gestion des biens immobiliers résidentiels au Togo. Face aux limites des pratiques manuelles encore dominantes (registres papier, relances téléphoniques, absence de traçabilité), une solution sur mesure a été développée pour faire face à ces problèmes.

La particularité de cette solution réside dans sa capacité à traiter, au sein d'une même architecture flexible, deux profils de gestion distincts : la location classique à durée indéterminée (cas des cours communes, immeubles et appartements) et la location de courte durée (appartements meublés et villas de type Airbnb).

Sur le plan technique, le système repose sur une architecture en trois couches :

- ✓ Un backend Spring Boot exposant une API REST sécurisée par des jetons JWT et s'appuyant sur une base de données relationnelle PostgreSQL gérée via Hibernate/JPA ;
- ✓ Une interface web Angular dédiée aux propriétaires pour la gestion complète de leurs biens, locataires, contrats de bail et tableaux de bord financiers ;
- ✓ Une application mobile Ionic/Angular destinée aux locataires de la location classique, leur offrant un accès transparent à leurs contrats et la possibilité de régler leur loyer en temps réel via l'intégration de la passerelle de paiement Mobile Money FedaPay.

L'implémentation de cette plateforme permet de centraliser 100 % des données de gestion, d'automatiser la génération et l'envoi de 95 % des avis d'échéance par notifications push, et de dématérialiser intégralement l'archivage des baux. La conception quant à elle repose sur la méthodologie UML, depuis les diagrammes de cas d'utilisation jusqu'au diagramme de classes et au modèle entité-relationnel.

TABLE DE MATIERES

DEDICACE	1
REMERCIEMENTS	2
AVANT - PROPOS	3
RESUME	4
TABLE DE MATIERES	5
LISTE DES TABLEAUX	7
LISTE DES FIGURES	8
INTRODUCTION GENERALE	11
Chapitre 1 : PRESENTATION DU CADRE DE LA FORMATION.....	12
1.1. Cadre de formation : Ipnet Institute of Technology (IIT)	13
1.1.1. Historique	13
1.1.2. Vision, mission et objectifs pédagogiques	14
1.1.3. Offre de formation	14
1.1.4. Organisation et implantation	14
Chapitre 2 : ETUDE PREALABLE DU PROJET.....	17
2.1. Présentation du projet	18
2.2. Problématique	18
2.3. Etude de l'existant	18
2.4. Critique de l'existant	19
2.5. Objectifs du projet	19
2.5.1. Objectif général	19
2.5.2. Objectif spécifiques	19
2.6. Propositions de solutions	20
2.7. Etude technique des solutions	21
2.8. Choix techniques des solutions	22
2.9. Evaluation financière du projet	23
2.10. Planning prévisionnel de réalisation du projet	24
Chapitre 3 : ANALYSE ET CONCEPTION.....	25
3.1. Présentation de la méthode d'analyse	26

3.2. Présentation du langage d'analyse	27
3.3. Démarches méthodologiques	28
3.4. Etude détaillée du projet	29
3.4.1. Diagramme de contexte	29
3.4.2. Diagramme de package	30
3.4.3. Diagramme de cas d'utilisation	31
3.4.4. Diagrammes d'activités	32
3.4.5. Diagrammes de séquences	36
3.4.6. Diagramme de classes	39
3.4.7. Diagrammes d'objets	41
3.4.8. Diagramme d'état-transition	42
3.4.9. Modèle Entité – Relationnel	44
Chapitre 4 : REALISATION	46
4.1. Réalisation	47
4.1.1. Les matériels utilisés	47
4.1.2. Les logiciels utilisés	47
4.1.3. Architectures matérielles et logicielles	48
4.1.4. Sécurité des applications	50
4.2. Scripts de création de la base de données	51
4.3. Quelques captures des interfaces et codes sources des applications	55
4.3.1. Captures des interfaces	55
4.3.2. Captures de codes sources	62
Chapitre 5 : GUIDE DE DEPLOIEMENT ET D'EXPLOITATION.....	64
5.1. Configurations matérielles et logicielles	65
5.2. Guide de déploiement	66
5.3. Guide d'exploitation	69
Chapitre 6 : GUIDE D'UTILISATION	71
6.1 Manuel opératoire du Propriétaire (interface web)	72
6.2 Manuel opératoire du locataire (interface mobile)	79
CONCLUSION GENERALE	82
BIBLIOGRAPHIE ET WEBOGRAPHIE	83

LISTE DES TABLEAUX

Table 1 : Tableau de l'évaluation financière	23
Table 2 : Tableau du planning du projet	24
Table 3 : Règle d'exploitation fedapay	70

LISTE DES FIGURES

Figure 1.1 : Organigramme de l'IPNET Institute of Technology.....	16
Figure 2.1 : Spring Boot.....	22
Figure 2.2 : Hibernate.....	22
Figure 2.3 : PostgreSQL.....	22
Figure 2.4 : Angular.....	22
Figure 2.5 : Ionic.....	22
Figure 2.6 : Fedapay.....	22
Figure 3.1 : Drawio.....	26
Figure 3.2 : PlantUml.....	26
Figure 3.3 : Diagramme de contexte.....	29
Figure 3.4 : Diagramme de package.....	30
Figure 3.5 : Diagramme de cas d'utilisation.....	31
Figure 3.6 : Diagramme d'activité de l'inscription.....	32
Figure 3.7 : Diagramme d'activité de l'authentification.....	33
Figure 3.8 : Diagramme d'activité de création d'un bail.....	34
Figure 3.9 : Diagramme d'activité du scheduler.....	35
Figure 3.10 : Diagramme d'activité du paiement par mobile Money.....	36
Figure 3.11 : Diagramme de séquence : création d'un bail.....	37
Figure 3.12 : Diagramme de séquence pour la Génération automatique des échéances.....	37
Figure 3.13 : Diagramme de séquence : vérification et le marquage des retards d'échéances.....	38
Figure 3.14 : Diagramme de séquence pour le paiement mobile money.....	38
Figure 3.15 : Diagramme de séquence : réception et traitement d'un webhook fedapay.....	39
Figure 3.16 : Diagramme de classes du système.....	40
Figure 3.17 : Diagramme d'objet.....	41
Figure 3.18 : Diagramme d'état-transition : Contrat de bai.....	42
Figure 3.19 : Diagramme d'état-transition : Echéance.....	43
Figure 3.20 : Diagramme d'état-transition : Unité de logement.....	43
Figure 3.21 : Diagramme d'état-transition : Notification.....	44
Figure 3.22 : Model entité relationel.....	45

Figure 4.1 : Architecture Logicielle et Materielle de la plateforme.....	49
Figure 4.2 : page web d'enregistrement.....	55
Figure 4.3 : page web de connection.....	55
Figure 4.4 : page web profil.....	56
Figure 4.5 : page web des gestion des propriétés.....	56
Figure 4.6 : formulaire d'ajout d'un nouveau bien.....	57
Figure 4.7 : formulaire d'ajout d'une unité de logement.....	57
Figure 4.8 : page web de gestion des unités de logements.....	58
Figure 4.9 : page web de gestion des baux.....	58
Figure 4.10 : page web de gestion des contrats.....	59
Figure 4.11 : formulaire d'ajout d'un contrat.....	59
Figure 4.12 : pages web de gestion des échéances.....	60
Figure 4.13 : formulaire d'ajout d'une échéance.....	60
Figure 4.14 : formulaire d'ajout d'un locataire.....	61
Figure 4.15 : Dashboard mobile & détail d'un contrat.....	61
Figure 4.16 : Code source du controlleur de bail.....	62
Figure 4.17 : Code source du fichier service angular.....	63
Figure 4.18 : Code source du fichier service Ionic.....	63
Figure 5.1 : fichier docker.....	66
Figure 6.1 : Ecran de d'authentification.....	72
Figure 6.2 : Menu du proprétaire.....	73
Figure 6.3 : Bouton d'ajout d'un bien.....	73
Figure 6.4 : Formulaire d'ajout d'un bien.....	74
Figure 6.5 : Bouton d'accès aux unités.....	74
Figure 6.6 : Bouton d'ajout d'une unité de logement.....	75
Figure 6.7 : Formulaire d'ajout d'une unité.....	75
Figure 6.8 : Bouton d'accès à la rubrique des baux.....	76
Figure 6.9 : Bouton d'ajout d'un locataire.....	76
Figure 6.10 : Formulaire d'enregistrement d'un locataire.....	76
Figure 6.11 : Onglet contrats	77
Figure 6.12 : Formulaire d'enregistrement d'un contrat.....	77
Figure 6.13 : Onglet des Echéances.....	78

Figure 6.14 : Bouton d'encaissement de loyer.....	78
Figure 6.15 : Bouton d'accès aux historiques des encaissements.....	78
Figure 6.16 : Bouton de résiliation d'un contrat.....	79
Figure 6.17 : Ecran d'authentification du mobile.....	79
Figure 6.18 : Dashboard locataire.....	80
Figure 6.19 : Détail d'un contrat.....	80
Figure 6.20 : Détail d'une échéance.....	81
Figure 6.21 : Notification.....	81

INTRODUCTION GENERALE

À l'instar de plusieurs métropoles en développement, le Togo, et plus particulièrement sa capitale Lomé, connaît un essor urbain sans précédent qui s'accompagne d'une multiplication rapide des biens résidentiels à caractères locatifs. Cette dynamique a fait émerger une diversité de besoins, allant de la location classique à durée indéterminée caractérisée par le phénomène des cours communes ou des immeubles à appartements jusqu'à la location de courte durée de type Airbnb destinée surtout aux étrangers. Cependant, face à cette croissance immobilière soutenue, le secteur souffre d'un décalage structurel majeur : les modes de gestion locative demeurent profondément artisanaux. L'immense majorité des bailleurs s'appuient sur des processus non digitalisés pour administrer leur patrimoine, limitant l'efficacité opérationnelle d'un marché en pleine expansion.

Cette gestion manuelle et informelle engendre des insuffisances critiques qui fragilisent l'équilibre des investissements et dégradent surtout les relations entre les parties prenantes (locataires & bailleurs). En s'appuyant sur des registres papier ou des tableurs isolés, les propriétaires s'exposent à des risques élevés de pertes de données. L'absence d'outils automatisés pour le suivi des échéances de baux entraîne inévitablement des détections tardives d'impayés et un recouvrement inefficace des loyers. De surcroît, le manque de traçabilité numérique des contrats et des historiques de versement prive le bailleur d'une visibilité financière en temps réel et prive le locataire d'un accès transparent à ses informations de paiement, créant un climat de méfiance réciproque.

Pour répondre à ces enjeux opérationnels et technologiques, ce projet propose la mise en place d'une plateforme logicielle (web et mobile) intégrée, sécurisée, dédiée à l'automatisation intégrale du cycle de gestion locative résidentielle. Conçu sur mesure pour s'adapter au contexte et aux infrastructures de paiement togolais, le système se structure autour d'une architecture moderne à trois couches combinant un backend Spring Boot (sécurisé par JWT), une interface d'administration web développée avec Angular pour les propriétaires, et une application mobile Ionic/Angular couplée à la passerelle Fedapay pour permettre aux locataires de régler leur loyer par Mobile Money. Cette solution centralisée se distingue par sa flexibilité multi-profils, capable de traiter distinctement les règles métiers de la location classique et le suivi dynamique des réservations et de la rentabilité des appartements de séjour Airbnb.

Le présent mémoire, qui retrace la démarche de conception et de réalisation de cette plateforme, est structuré en six chapitres. Le premier chapitre présente le cadre de notre formation au sein de Ipnet Institute of Technology (IIT). Le deuxième chapitre est consacré à l'étude préalable du projet, détaillant l'analyse de l'existant, la problématique, les objectifs ainsi que l'étude d'opportunité financière et technique. Le troisième chapitre expose la phase d'analyse et de conception logicielle formalisée par les diagrammes UML et le Modèle Entité-Relationnel. Le quatrième chapitre décrit la réalisation technique, l'environnement technologique retenu, la mise en œuvre de la sécurité et présente les captures clés des applications et codes sources. Le cinquième chapitre fournit les guides de déploiement et d'exploitation nécessaires à la mise en production du système. Enfin, le sixième chapitre fait office de guide d'utilisation détaillant les manuels opératoires pour chaque profil d'acteur, avant de clore ce document par une conclusion générale et l'exposé des perspectives d'évolution de notre plateforme.

Chapitre 1 : PRESENTATION DU CADRE DE FORMATION

1.1. Cadre de formation : Ipnnet Institute of Technology (IIT)

Fondé à Lomé (Togo), iPNet Institute of Technology (IIT) est un établissement d'enseignement supérieur de référence, spécialisé dans les sciences et technologies de l'information et de la communication. Guidé par sa vision centrale d'« adéquation formation-emploi », l'institut s'est donné pour mission majeure de former la prochaine génération de professionnels des technologies de l'information ("*Next Generation IT Professionals*") et de transformer ses apprenants en créateurs de valeurs et futurs entrepreneurs de l'écosystème numérique africain. Signe de sa forte dynamique d'innovation pédagogique, l'institut a d'ailleurs été primé comme l'Université la plus innovante du Togo.

L'IIT se distingue par son rayonnement panafricain, accueillant des étudiants issus d'une vingtaine de pays d'Afrique. Son modèle d'enseignement moderne est conçu à mi-chemin entre une école d'ingénieurs classique et un incubateur de startups. Pour garantir un apprentissage pratique et immersif, l'établissement est doté d'installations de pointe, intégrant notamment un réseau informatique grandeur nature accessible en permanence, permettant aux étudiants de mener des travaux pratiques approfondis et de concevoir des projets technologiques concrets. En plus de ses cursus de formation classiques, l'IIT met un accent fort sur l'excellence internationale à travers des partenariats stratégiques avec les plus grandes firmes informatiques mondiales, incitant ses étudiants à décrocher des certifications reconnues mondialement durant leur parcours.

1.1.1. Historique

L'histoire de l'IIT prend racine dans le groupe IPNET EXPERTS SA, créé en juin 2003 comme intégrateur de solutions informatiques spécialisé notamment en sécurité des systèmes d'information, et actif dans plusieurs pays d'Afrique de l'Ouest et du Centre. Dans la continuité de cette expertise technique, l'institut a été officiellement créé en octobre 2016 sous la forme d'une grande école d'informatique et de gestion de projets, avant d'ouvrir ses portes à Lomé en 2017, avec pour ambition de proposer une formation résolument pratique et professionnalisante, fondée sur des travaux pratiques et des projets réels.

En quelques années seulement, l'établissement a connu une croissance soutenue, passant d'une vingtaine d'étudiants à sa création à environ 500 apprenants en 2024, avec l'ambition affichée d'atteindre 1 400 étudiants à l'horizon 2027. Cette dynamique a été récompensée par plusieurs distinctions obtenues en 2022, dont la « Palme Internationale de l'Université la plus Innovante du Togo » et le « Prix Africain du Mérite et de l'Excellence », décerné au Rwanda.

La gouvernance de l'institut s'appuie sur une équipe dirigeante pluridisciplinaire, conduite par M. Pawou P. BATANA, Co-fondateur et Directeur Général, ingénieur en cybersécurité et en télécommunications, également gestionnaire de projets. Il est entouré de M. Panawé BATANADO, Co-fondateur et Conseiller du Directeur Général, de M. Ferdinand Alou P. BATANA, Directeur Académique et Directeur de l'Innovation et des Systèmes d'Information,

assisté de M. Paul SEDJRO, Adjoint au Directeur Académique, ainsi que de M. Samtoug AGORO, Directrice Administrative, Financière et Comptable par intérim, de M. Habré BOUWIKI, en charge du département commercial et de la formation, et de Mme Essoyomèwè ADAMAGNON, responsable de la gestion des projets, des bourses et aides financières.

1.1.2. Vision, mission et objectifs pédagogiques

Au-delà de cette vocation générale, l'institut traduit sa mission en objectifs pédagogiques concrets : former des diplômés aptes à la résolution de problèmes réels, notamment grâce à un réseau informatique grandeurs nature accessible en permanence pour les travaux pratiques ; rapprocher les étudiants du monde professionnel à travers les Journées Portes Ouvertes Projets Étudiants (JPOPE), organisées chaque année, ainsi qu'un site web permettant aux entreprises de consulter directement les solutions développées par les équipes étudiantes ; et encourager l'esprit entrepreneurial en initiant les apprenants aux fondamentaux de la gestion de projets, en complément de leur spécialité technique initiale.

1.1.3. Offre de formation

L'IIT propose un cycle long de trois ans (six semestres) sanctionné par une Licence Professionnelle, ainsi qu'un parcours en cours du soir de quinze mois destiné aux personnes en activité. Le cycle long se décline en huit filières : Cybersécurité, Réseaux, Systèmes & Sécurité, Génie Logiciel, Développement Web & Mobile, Webdesign, Infographie & Multimédia, Informatique et Gestion, Mathématiques et Informatique, ainsi que Génie Électrotechnique & Énergies Renouvelables. L'institut offre par ailleurs plusieurs programmes de Master (Génie Logiciel, Réseaux, Systèmes & Sécurité, Cybersécurité, Gestion Stratégique de Projets, Intelligence Artificielle, etc.), ainsi qu'une offre étendue de certifications professionnelles couvrant les réseaux et la cybersécurité, l'infographie et le multimédia, le développement web et la gestion de projet, permettant aux étudiants de renforcer leur employabilité tout au long de leur parcours.

C'est dans ce cadre d'excellence, et plus spécifiquement au sein de la filière Licence Professionnelle en Informatique, spécialité Génie Logiciel, que s'inscrit le développement de notre application. Ce cursus s'attache à doter les étudiants d'une solide expertise dans le cycle de vie logiciel (analyse, modélisation UML, architectures logicielles et sécurité). La réalisation de notre plateforme web et mobile s'appuie directement sur les standards technologiques et la rigueur d'ingénierie inculqués tout au long de cette formation à l'IIT.

1.1.4. Organisation et implantation

Sur le plan organisationnel, l'IIT s'appuie sur une gouvernance structurée autour de la Direction Générale, à laquelle sont rattachés un Conseil Scientifique et un Conseil Pédagogique, ainsi que quatre grandes directions : la Direction Académique, la Direction

Commerciale, Marketing, Communication et de la Formation Continue, la Direction de l'Innovation et des Systèmes d'Information, et la Direction Administrative, Financière et Comptable. Chacune de ces directions chapeaute des départements et services spécialisés, garantissant un pilotage rigoureux des volets pédagogique, technique, commercial et administratif de l'institut (Figure 1.1).

L'établissement est implanté à Agbalépédo Lossossimè, à Lomé (Togo), et demeure joignable via son site web (www.ipnetinstitute.com) ainsi que par téléphone et courrier électronique. C'est dans ce cadre institutionnel structuré, résolument tourné vers l'innovation et la pratique professionnelle, que s'est déroulée notre formation et qu'a été conçu et réalisé le présent projet de fin de cycle.

Organigramme
IPNet Institute of Technology
Mars 2026

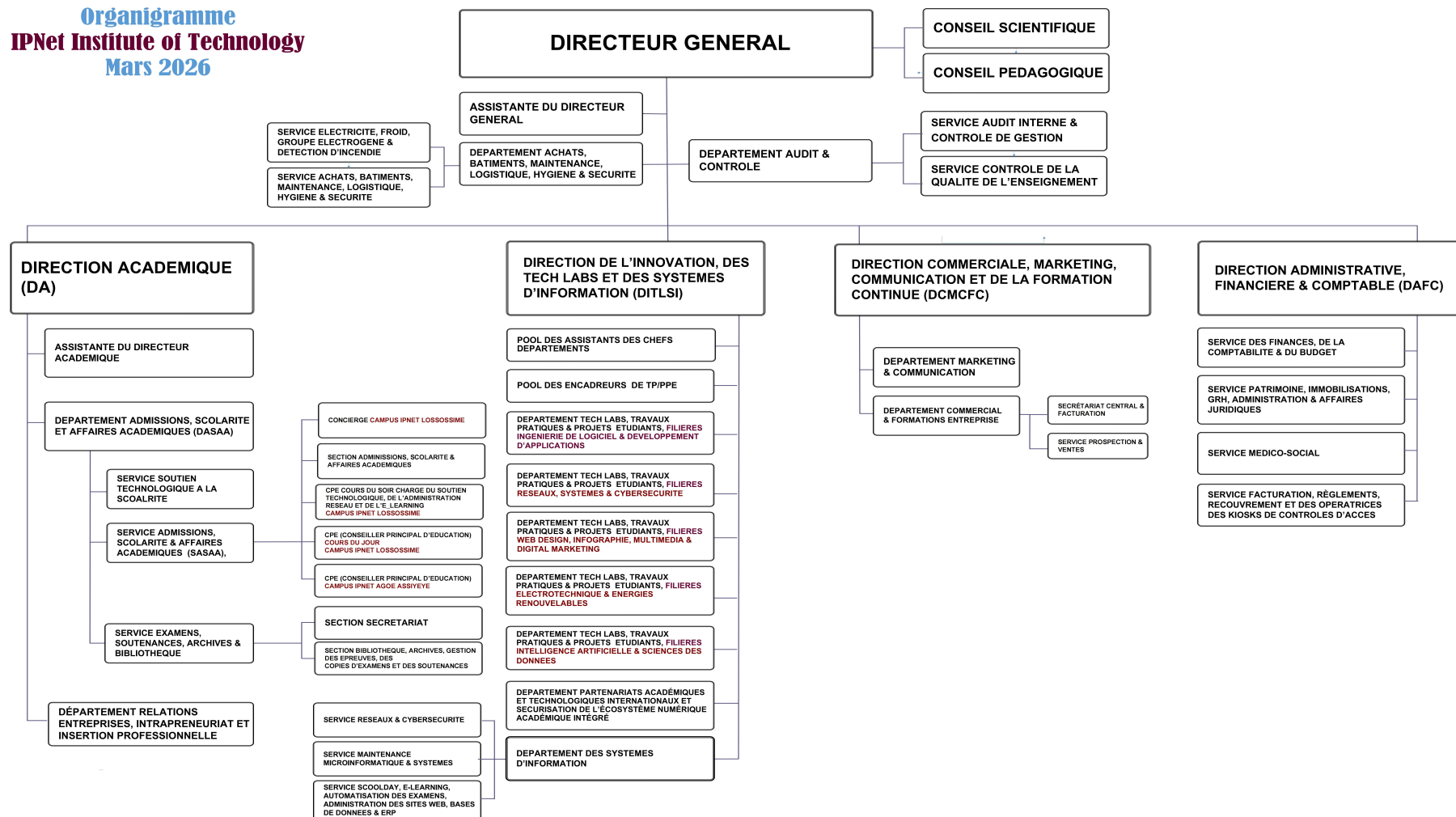


Figure 1.1 : Organigramme de l'IPNET Institute of Technology

Chapitre 2 : ETUDE PREALABLE DU PROJET

2.1. Présentation du projet

Le projet consiste en la mise en place d'une plateforme numérique (web et mobile) dédiée à la gestion des biens immobiliers résidentiels au Togo. Cette plateforme s'adresse aux propriétaires d'immeuble, maisons à louer et couvre deux profils de gestion distincts : la location classique à durée indéterminée (cours communes, immeubles, appartements) et la location de courte durée de type Airbnb (appartements meublés et villas de séjour).

Le système repose sur une architecture en trois couches : un backend Spring Boot exposant une API REST sécurisée par JWT, une interface web développée en Angular destinée aux propriétaires (gestion des biens, des locataires, des contrats de bail, des échéances et des paiements), et une application mobile développée avec Ionic/Angular destinée aux locataires, leur permettant de consulter leur contrat, suivre leurs échéances et régler leur loyer par Mobile Money. L'objectif est de remplacer les pratiques manuelles actuelles par un outil centralisé, traçable et automatisé.

2.2. Problématique

La question centrale de notre étude est de savoir comment concevoir et implémenter une architecture logicielle capable d'automatiser le cycle de gestion immobilière tout en garantissant la fiabilité des données, mais aussi capable de s'adapter à tout type de location immobilière (location classique et location de courte durée).

De cette interrogation découlent les défis suivants :

- ❓ Comment concevoir une base de données et une architecture logicielle suffisamment flexibles pour gérer des biens de natures différentes (immeubles, appartements(unités), boutique) sans avoir à redévelopper le système pour chaque cas ?
- ❓ Comment garantir une cohérence parfaite des informations entre une interface d'administration web (Spring Boot/Angular) et une interface mobile destinée aux locataires, pour assurer une gestion en temps réel ?
- ❓ Comment structurer un système de notifications et capable de traiter les échéances de contrats et les relances de loyers impayés ?

Pour répondre à ces enjeux, le système se limite fonctionnellement à deux profils de gestion distincts : la location classique et la location de courte durée, dont les détails sont fournis dans le cahier des charges fonctionnel (section 2.6).

2.3. Etude de l'existant

À Lomé comme dans le reste du Togo, la gestion locative résidentielle repose encore très largement sur des pratiques manuelles et informelles. Les propriétaires tiennent généralement un registre papier ou, au mieux, un tableur (Excel) pour consigner l'identité des locataires, les montants de loyer et les dates d'échéance. Le suivi des paiements se fait souvent de mémoire ou par carnet de quittances manuscrites, et la communication des rappels d'échéance passe par des appels téléphoniques ou des messages WhatsApp envoyés au cas par cas.

Les contrats de bail, lorsqu'ils « existent » sous forme écrite, sont conservés en version papier unique, sans copie de sauvegarde ni possibilité de consultation à distance. Enfin, dans le segment de la location de courte durée (Airbnb local), la prise de réservation et le calcul du tarif (nombre de nuits, réductions éventuelles) sont effectués manuellement, sans calendrier centralisé de disponibilité.

2.4. Critique de l'existant

Cette gestion artisanale présente plusieurs limites structurelles :

1. L'absence de centralisation des données expose à la perte d'information (carnets perdus, fichiers Excel corrompus ou non sauvegardés) et favorise les doublons et incohérences entre plusieurs supports.
2. Le suivi manuel des échéances entraîne une détection tardive des impayés ; un retard de quelques jours peut se transformer en créance difficile à recouvrer faute de relance immédiate.
3. L'absence de traçabilité numérique des contrats et des paiements fragilise la relation de confiance entre bailleurs et locataires, chaque partie ne disposant pas toujours d'une preuve facilement accessible des transactions effectuées (souvent ces preuves sont de simple signature sur papier conservée uniquement chez le bailleur) .
4. Enfin, la gestion manuelle ne permet aucune vision consolidée et instantanée (taux d'occupation, rentabilité par bien, impayés en cours), ce qui prive le propriétaire d'un outil d'aide à la décision.

2.5. Objectifs du projet

2.5.1. Objectif général

L'objectif général de ce projet est de concevoir et réaliser une application (web/mobile) dédiée à l'automatisation de la gestion locative, afin de garantir la rentabilité des investissements immobiliers.

2.5.2. Objectif spécifiques

1. Automatisation & gestion

- a. Centralisation des données : mettre en place un référentiel unique centralisant 100 % des données relatives aux biens immobiliers et aux locataires, pour éliminer les doublons d'information.
- b. Automatisation administrative : automatiser la génération des avis d'échéance de sorte que 95 % de ces avis soient émis à chaque cycle de facturation (chaque mois).
- c. Dématérialisation : atteindre un taux de 100 % de numérisation et d'archivage sécurisé pour les contrats de bail.

2. Suivi et communication

- a. Réactivité des alertes : configurer un système de notification «push mobile» capable de notifier les locataires en temps réel (avis d'échéance de paiement, confirmation de paiement etc...).
- b. Amélioration du recouvrement : réduire le taux de retard de paiement sur les six premiers mois grâce aux notifications de relances.

2.6. Propositions de solutions

Face aux limites identifiées, la solution proposée est une plateforme numérique structurée autour de deux profils de gestion, chacun doté de modules fonctionnels spécifiques.

A. Profil A — Location classique (bail à durée indéterminée)

- Gestion des biens (immeubles et appartements) : enregistrement, modification, consultation avec filtres.
- Gestion des locataires et des contrats de bail : création, renouvellement, résiliation.
- Loyers et quittances : enregistrement des paiements, génération automatique de quittances, calcul des impayés s'il y a lieu.
- Notifications automatiques : alertes d'échéance, relances de loyers impayés, confirmations de paiement (push mobile).
- Tableau de bord et rapports : taux d'occupation, revenus nets, impayés en cours.

B. Profil B — Appartements de séjour (location de courte durée)

- Gestion des appartements meublés : enregistrement, mise en maintenance temporaire, historique des séjours.
- Gestion des réservations : enregistrement manuel, calcul automatique du montant (tarif × nuits, avec réductions), clôture.
- Suivi des dépenses par appartement et calcul de rentabilité nette.
- Taux d'occupation et calendrier visuel (jours occupés / libres / en maintenance).
- Tableau de bord : chiffre d'affaires total, dépenses, bénéfice net, et filtres dynamiques.

C. Acteurs et interfaces

- Propriétaire : accès complet via l'interface web Angular (tous les modules des deux profils).
- Locataire (Profil A uniquement) : accès limité, en lecture et en paiement, via l'application mobile Ionic/Angular (contrat, historique de paiements, notifications).

- Client de séjour (Profil B) : pas d'interface dédiée .

D. Contraintes et exigences non fonctionnelles

- Sécurité : authentification JWT et HTTPS obligatoires ; isolation stricte des données entre propriétaires.
- Performance : temps de réponse API raisonnable, même en cas de pics de connexion.
- Maintenabilité : architecture en couches (Controller, Service, Repository) avec code documenté.
- Ergonomie mobile : application Ionic/Angular compatible Android 8.0 & iOS et supérieur.

2.7. Etude technique des solutions

Avant d'arrêter le choix d'une solution sur mesure, trois pistes ont été comparées :

? Solution 1 :

Description : Adopter un logiciel de gestion immobilière existant sur le marché (SaaS international).

Avantages : Mise en œuvre rapide, support éditeur

Limites : Coût d'abonnement récurrent en devises, absence de Mobile Money local (Flooz/T-Money), faible adaptabilité au contexte togolais (deux profils de gestion, indexation locale).

? Solution 2 :

Description : Développer une application web unique, sans application mobile dédiée.

Avantages : Périmètre de développement réduit.

Limites : Aucun accès direct du locataire à son contrat et à ses échéances ; dépendance totale au propriétaire pour toute information, ce qui ne répond pas à l'objectif de transparence.

? Solution 3 (retenue) :

Description : Développer une plateforme sur mesure combinant une application web (propriétaires) et une application mobile (locataires), avec intégration Mobile Money locale.

Avantages : Adaptée aux deux profils de gestion, maîtrise totale des données, intégration native du paiement Mobile Money (Fedapay/Moov Money/T-Money), évolutivité.

Limites : Délai et effort de développement plus importants, nécessite une maîtrise de plusieurs technologies.

La Solution 3 a été retenue car elle est la seule à répondre simultanément aux trois exigences majeures identifiées dans la problématique : flexibilité multi-profils, cohérence temps réel

entre web et mobile, et autonomie du système de notification/recouvrement adapté au contexte togolais (paiement Mobile Money notamment).

2.8. Choix techniques des solutions

A. Backend : Spring Boot (Java)

Spring Security a été utilisé pour gérer l'authentification (JWT) et les autorisations par rôle (PROPRIETAIRE, LOCATAIRE). L'architecture en couches (Controller, Service, Repository) garantit un code propre et maintenable, tandis que l'intégration avec Hibernate/JPA simplifie la gestion de la base de données relationnelle (PostgreSQL).



Figure 2.8 : Spring Boot



Figure 2.8 : Hibernate



Figure 2.8 : PostgreSQL

B. Frontend web : Angular (TypeScript)

Angular a été retenu pour son caractère de framework complet ("tout-en-un"), offrant un cadre et une structure stricte. Le mode Single Page Application (SPA) garantit une expérience utilisateur fluide, sans rechargement de page à chaque action.



Figure 2.8 : Angular

C. Mobile : Ionic/Angular (TypeScript)

Le choix d'Ionic/Angular pour l'application mobile répond à la volonté d'unifier le développement frontend. En partageant le même framework TypeScript qu'Angular, les composants et la logique métier sont en partie réutilisables entre le web et le mobile. Ionic génère des applications hybrides compatibles Android (et iOS) depuis une base de code unique, réduisant les coûts de maintenance.



Figure 2.8 : Ionic

D. Paiement Mobile Money : Fedapay

L'intégration de Fedapay permet de proposer aux locataires un paiement de loyer dématérialisé via Flooz ou T-Money, avec confirmation automatique du paiement par webhook sécurisé (vérification de signature HMAC-SHA256), évitant toute saisie manuelle par le propriétaire.



Figure 2.8 : Fedapay

2.9. Evaluation financière du projet

Table 1 : Tableau de l'évaluation financière

Poste de dépense	Description	Estimation
Ressources humaines	Temps de conception et de développement	3 à 4 mois
Hébergement backend	Render plan (version Free) utilisé pendant le développement et la démonstration	O FCFA
Hébergement web & mobile	Build et déploiement de l'interface web et de l'app mobile	O FCFA
Frais de transaction Mobile Money	Commission Fedapay par transaction (sandbox : gratuit ; production : 2,5 à 3,5 % de frais de prélèvement par transaction)	O FCFA (Mode Sandbox)
Stockage de fichiers (avatars, documents)	Supabase Storage	O FCFA

2.10. Planning prévisionnel de réalisation du projet

Table 2 : Tableau du planning du projet

Phase	Contenu	Durée estimée
1. Étude préalable	Contexte, problématique, cahier des charges	1 mois
2. Analyse et conception	Diagrammes UML (cas d'utilisation, classes, séquences)	2 semaines
3. Développement backend	API Spring Boot, sécurité JWT, modèles de données	3 semaines
4. Développement web	Interface Angular propriétaire (biens, baux, échéances)	2 semaines
Développement mobile	Application Ionic/Angular locataire	1 semaine
6. Intégration paiement & finalisation Airbnb	FedaPay Mobile Money, module Airbnb,	1 à 2 semaines
7. Rédaction et finalisation du mémoire	Chapitres 1, 3 à 6, relecture	1 mois

Chapitre 3 : ANALYSE ET CONCEPTION

3.1. Présentation de la méthode d'analyse

Dans le cadre du développement de logiciels d'entreprise modernes, le choix d'une approche méthodologique rigoureuse est une étape fondamentale pour garantir la qualité, la maintenabilité et l'évolutivité du système à concevoir. Pour ce projet de mise en place d'une plateforme web et mobile de gestion immobilière, nous avons adopté une approche orientée objet pilotée par les cas d'utilisation, pleinement conforme aux bonnes pratiques de l'ingénierie logicielle moderne enseignées à l'IIT.

Cette démarche place l'utilisateur au centre du processus de développement. En se focalisant sur les cas d'utilisation, elle permet de capturer avec précision les besoins fonctionnels réels des différents acteurs (propriétaires et locataires) et de les traduire de manière transparente en structures logicielles (classes, services, contrôleurs). L'alignement de cette approche orientée objet avec l'architecture en couches de notre backend Spring Boot et l'organisation modulaire d'Angular/Ionic assure une transition fluide et sans rupture de concepts entre la phase d'analyse et la phase de codage.

Afin de formaliser, documenter et communiquer efficacement l'architecture de la plateforme, la modélisation a été entièrement réalisée à l'aide du langage UML (Unified Modeling Language) dans sa version 2.x. UML s'impose ici comme le standard de fait de l'industrie pour visualiser, spécifier et construire les artefacts d'un système logiciel.

Pour la production et la maintenance des différents diagrammes (contexte, cas d'utilisation, activités, séquences, classes et objets), nous avons combiné l'utilisation de deux outils de modélisation de premier plan :

- ✓ Draw.io : Un outil de diagrammation polyvalent et intuitif, particulièrement adapté pour concevoir les diagrammes structurels de haut niveau (contexte, packages) et assurer une clarté visuelle optimale pour la lecture du mémoire.
- ✓ PlantUML : Un outil de modélisation basé sur la syntaxe *Component-Driven Text-to-Diagram*. En nous permettant de générer des diagrammes comportementaux complexes (séquences et état-transition) à partir de scripts textuels simples, PlantUML a garanti une précision technique rigoureuse, une traçabilité du code de conception et une facilité de mise à jour tout au long des itérations du projet.

L'association de cette méthode pilotée par les cas d'utilisation et de la rigueur d'UML 2.x constitue la fondation de l'étude détaillée présentée dans les sections suivantes de ce chapitre.



Figure 3.1: Drawio



Figure 3.2: PlantUml

3.2. Présentation du langage d'analyse

La formalisation des spécifications d'un système logiciel complexe exige un langage commun, précis et universellement reconnu. Pour la conception de notre plateforme de gestion immobilière, le choix s'est naturellement porté sur UML (Unified Modeling Language), ou Langage de Modélisation Unifié.

UML est un langage de modélisation visuel basé sur un méta-modèle, devenu le standard international incontournable de l'industrie logicielle sous l'égide de l'OMG (Object Management Group). Loin d'être une méthode de développement rigide, UML se définit comme un langage graphique qui fournit un ensemble d'outils et de concepts permettant de spécifier, visualiser, construire et documenter les artefacts d'un système informatique, en adoptant une perspective orientée objet. Son rôle est crucial : il sert de passerelle entre l'expression des besoins métiers des utilisateurs et l'implémentation technique par le développeur, réduisant ainsi les risques de mauvaise interprétation fonctionnelle.

UML 2.x s'articule autour de quatorze diagrammes officiels, classés en deux grandes catégories : les diagrammes structurels (statiques) et les diagrammes comportementaux (dynamiques). Pour des raisons d'efficacité, de pertinence et de conformité aux objectifs fonctionnels de notre application, nous avons sélectionné un sous-ensemble de diagrammes clés :

1. Les diagrammes structurels retenus :

- ✓ Diagramme de contexte : Il permet de délimiter le périmètre du système en identifiant les frontières de la plateforme et ses interactions globales avec l'environnement extérieur.
- ✓ Diagramme de package : Il structure l'architecture globale de l'application en regroupant les éléments logiques (modules métiers), assurant ainsi une modularité propre à Spring Boot et Angular.
- ✓ Diagramme de classes : C'est le cœur de la modélisation structurelle. Il présente l'organisation statique du système en spécifiant les classes (Biens, Contrats, Utilisateurs, etc.), leurs attributs, leurs méthodes ainsi que les types de relations (association, héritage, composition) qui les lient. Il préfigure directement la structure de notre base de données PostgreSQL et de nos entités JPA.
- ✓ Diagramme d'objets : Utilisé de manière ponctuelle, il permet d'illustrer des exemples concrets d'instances de classes à un instant T, facilitant la compréhension de structures de données spécifiques (comme l'agencement d'un immeuble en plusieurs appartements/unités).

2. Les diagrammes comportementaux retenus :

- ✓ Diagrammes de cas d'utilisation : Ils schématisent les fonctionnalités de la plateforme du point de vue des acteurs (Propriétaire, Locataire), formalisant ainsi les attentes de chaque profil d'utilisateur vis-à-vis du système.

- ✓ Diagrammes d'activités : Ils modélisent le flux de contrôle et le comportement séquentiel ou parallèle d'un traitement métier complexe, tel que l'algorithme de calcul automatique d'une réservation Airbnb ou le cycle de facturation mensuelle.
- ✓ Diagrammes de séquences : Ils décrivent l'aspect dynamique du système en mettant en évidence l'ordre chronologique des échanges de messages entre les différents objets (ou couches de l'application : Controller, Service, Repository) pour accomplir un cas d'utilisation précis, comme l'authentification sécurisée par jeton JWT.
- ✓ Diagramme d'état-transition : Il illustre les différents états par lesquels peut passer un objet métier volatil au cours de son cycle de vie (par exemple, un appartement qui passe de l'état "Libre" à "Occupé", "En maintenance" ou une réservation "En attente", "Confirmée", "Clôturée").

Ce choix ciblé de diagrammes offre une vue à 360 degrés de notre application, garantissant une base rigoureuse pour l'étape de réalisation technique.

3.3. Démarches méthodologiques

La conduite d'un projet informatique nécessite l'adoption d'un cadre méthodologique rigoureux, capable d'orienter efficacement les efforts de développement tout en minimisant les risques de dérive par rapport aux objectifs fonctionnels. Pour la mise en place de notre plateforme de gestion immobilière, nous avons suivi un cycle de vie logiciel structuré, découpé en quatre grandes étapes interdépendantes :

1. Le recueil et l'analyse des besoins : Cette première phase a consisté à étudier l'état de l'art et les pratiques manuelles au Togo afin de dresser un inventaire précis des exigences fonctionnelles et non fonctionnelles (définition des profils Location classique et Airbnb, choix du paiement Mobile Money).
2. La modélisation : En nous appuyant sur le langage UML 2.x, cette étape a permis de formaliser l'architecture conceptuelle, de cartographier les cas d'utilisation et de structurer le modèle relationnel de la base de données avant l'écriture du code.
3. L'implémentation : Cette phase a concrétisé la conception à travers le développement en couches du backend Spring Boot, la création de l'interface d'administration web avec Angular, et la réalisation de l'application mobile avec Ionic/Angular.
4. Les tests et la validation : Enfin, une phase de tests (intégration des API et de validation de la sandbox FedaPay) a été menée pour garantir, la sécurité (JWT) et la fluidité des flux opérationnels entre les interfaces.

Justification de l'approche itérative et incrémentale

Pour orchestrer ce cycle structuré, nous avons fait le choix stratégique de retenir une approche itérative et incrémentale, plutôt qu'un modèle rigide en cascade. Ce choix est pleinement justifié par deux contraintes majeures inhérentes à notre cadre de travail :

- ✓ Un projet mené en solo : En tant qu'unique concepteur et développeur de cette solution multi-plateforme, il était indispensable de fragmenter la complexité du système. L'approche incrémentale a permis de construire le logiciel brique par brique. Nous avons

d'abord stabilisé les fondations (le backend et la base de données), puis développé l'incrément de la location classique (Profil A) incluant l'application mobile, avant de déployer l'incrément dédié à Airbnb (Profil B).

- ✓ Un calendrier restreint (3 à 4 mois) : Disposant d'un délai de réalisation fixé à quelques mois selon notre planning prévisionnel, ce modèle dynamique nous a offert la flexibilité nécessaire pour réajuster rapidement le tir. Chaque itération se soldait par un livrable fonctionnel et testable, réduisant la complexité et garantissant la livraison d'une plateforme stable, pleinement opérationnelle et sécurisée.

3.4. Etude détaillée du projet

Cette partie se focalise sur la formalisation de l'ensemble des spécifications fonctionnelles et techniques nécessaires à la réalisation de la plateforme. Son objectif est de modéliser avec précision les interactions, les flux de données et les règles métiers afin de répondre à notre problématique posée.

3.4.1. Diagramme de contexte

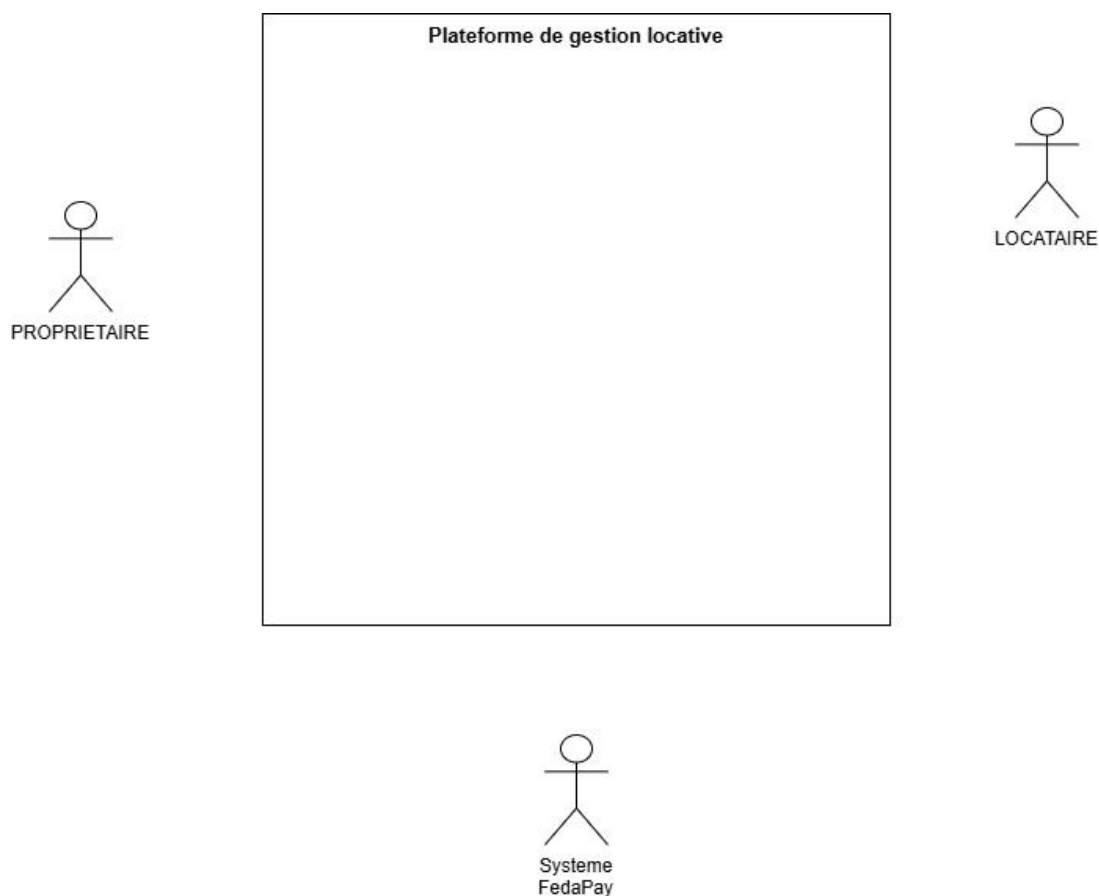


Figure 3.3 : Diagramme de contexte

Ce diagramme délimite le périmètre du système. Il identifie deux acteurs principaux en interaction avec la plateforme :

1. le Propriétaire, qui accède au système via l'interface web,
2. Locataire, qui interagit via l'application mobile.
3. Un acteur externe, FedaPay, qui représente la passerelle de paiement Mobile Money.

Le système reçoit les actions des acteurs et leur retourne des informations en temps réel.

3.4.2. Diagramme de package

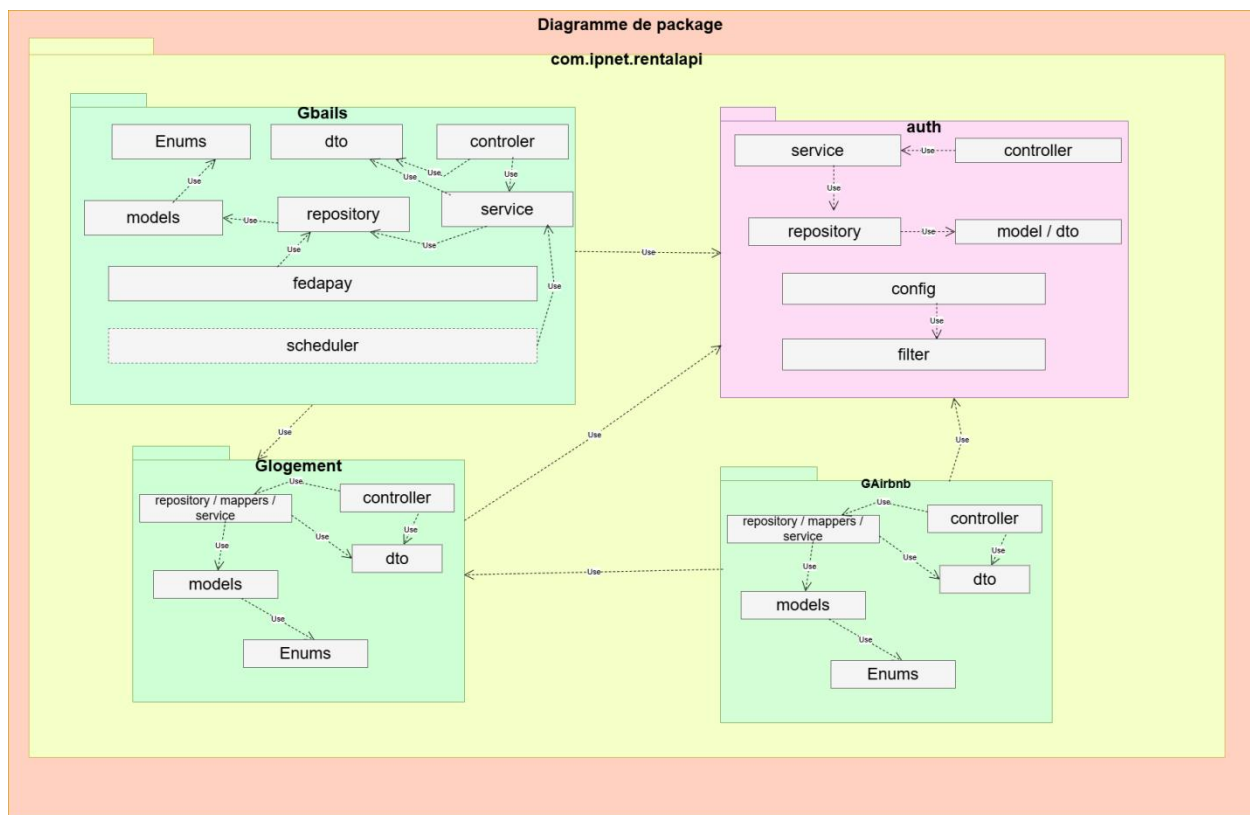


Figure 3.4 : Diagramme de package

Ce diagramme de package illustre l'organisation modulaire du système en grandes couches fonctionnelles à savoir :

le package Authentification, le package Gestion des biens(logements) et des bails, le package Gestion des Airbnb.

Cette organisation reflète l'architecture en couches (Controller / Service / Repository) adoptée côté backend.

3.4.3. Diagramme de cas d'utilisation

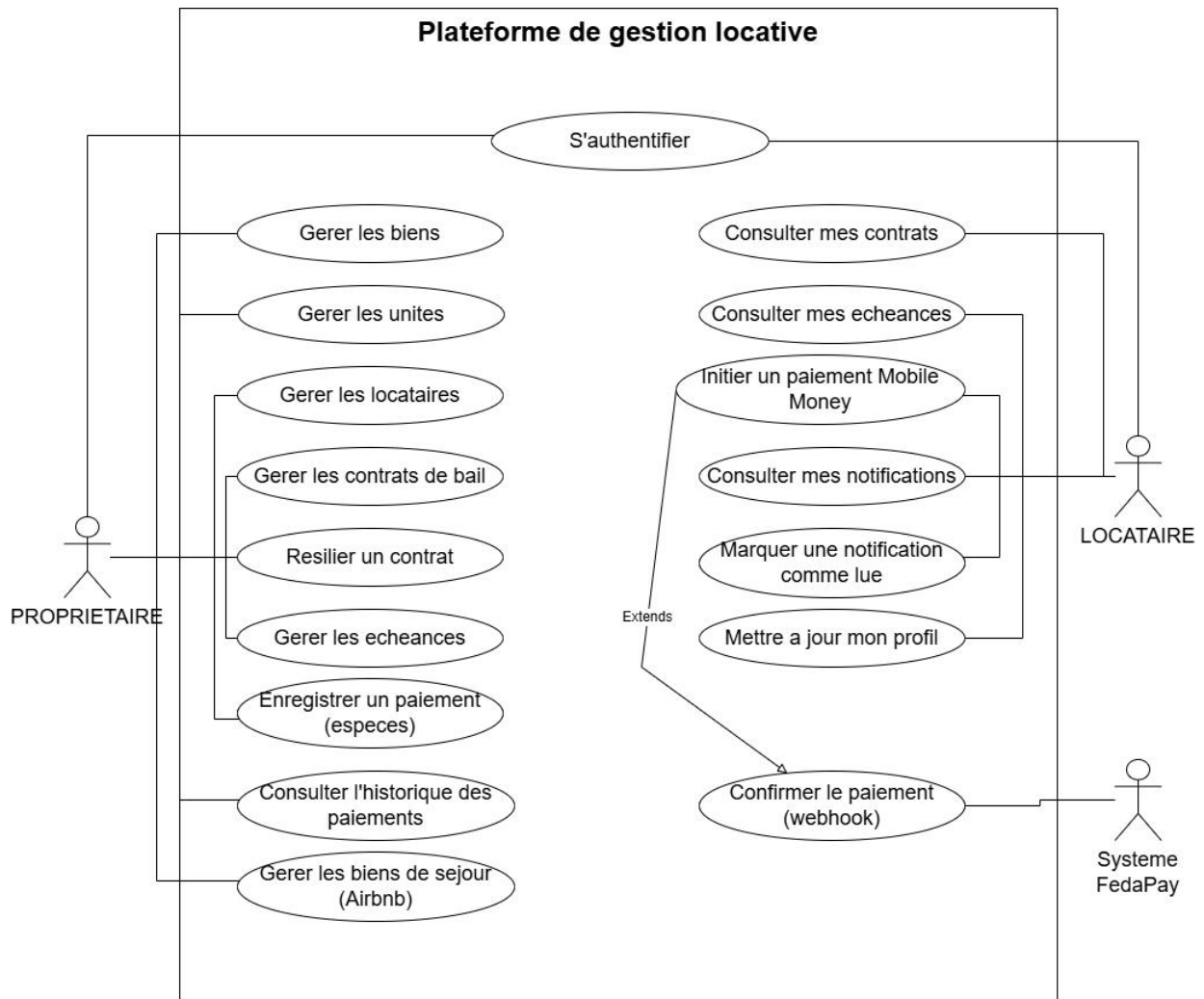


Figure 3.5 : Diagramme de cas d'utilisation

Ce diagramme de cas d'utilisation décrit les interactions fonctionnelles entre les acteurs et le système. Le Propriétaire dispose d'un accès étendu couvrant la gestion des biens, des locataires, des contrats de bail, des échéances et du tableau de bord. Le Locataire dispose d'un accès restreint : consultation de son contrat, suivi de ses échéances et initiation d'un paiement par Mobile Money.

3.4.4. Diagrammes d'activités

Les diagrammes d'activités modélisent les flux comportementaux des processus clés du système. Chaque diagramme est présenté sous forme de couloirs (swimlanes) afin d'identifier clairement la responsabilité de chaque acteur ou composant à chaque étape du flux.

1. Le diagramme ci-dessous présente le processus d'inscription.

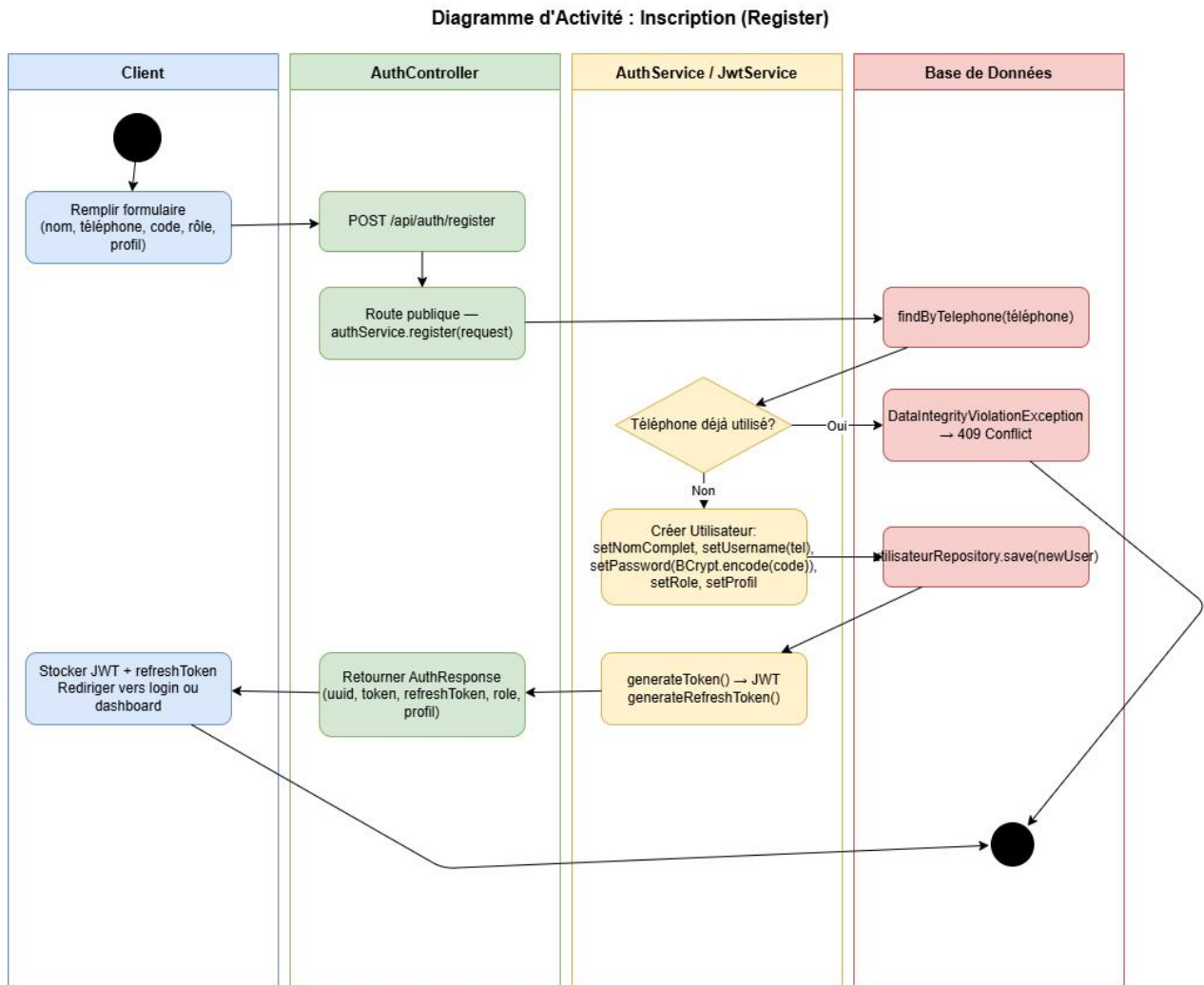


Figure 3.6 : Diagramme d'activité de l'inscription

2. Le diagramme ci-dessous présente le processus d'authentification.

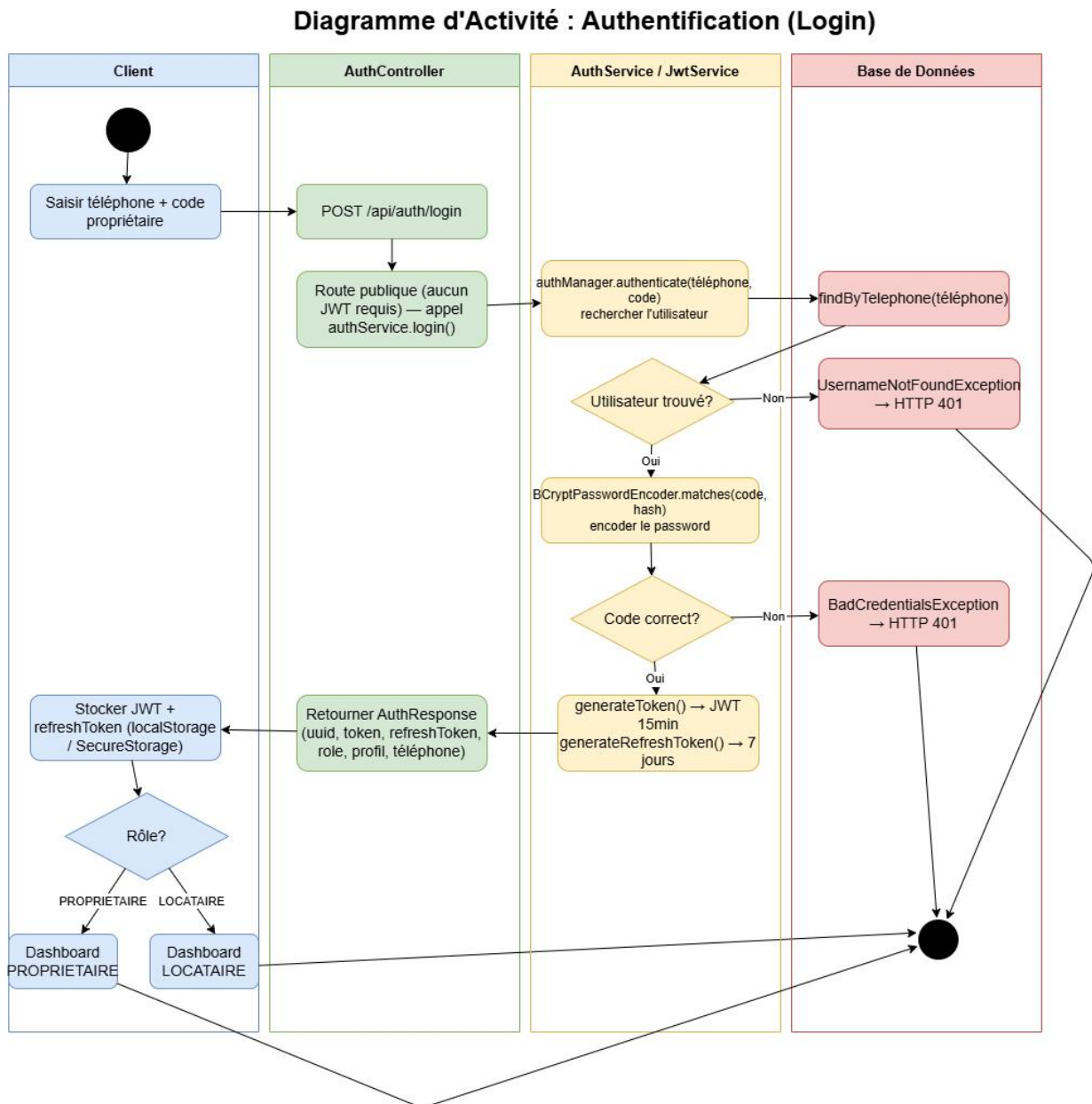


Figure 3.7 : Diagramme d'activité de l'authentification

3. Le diagramme ci-dessous présente le processus de création d'un bail.

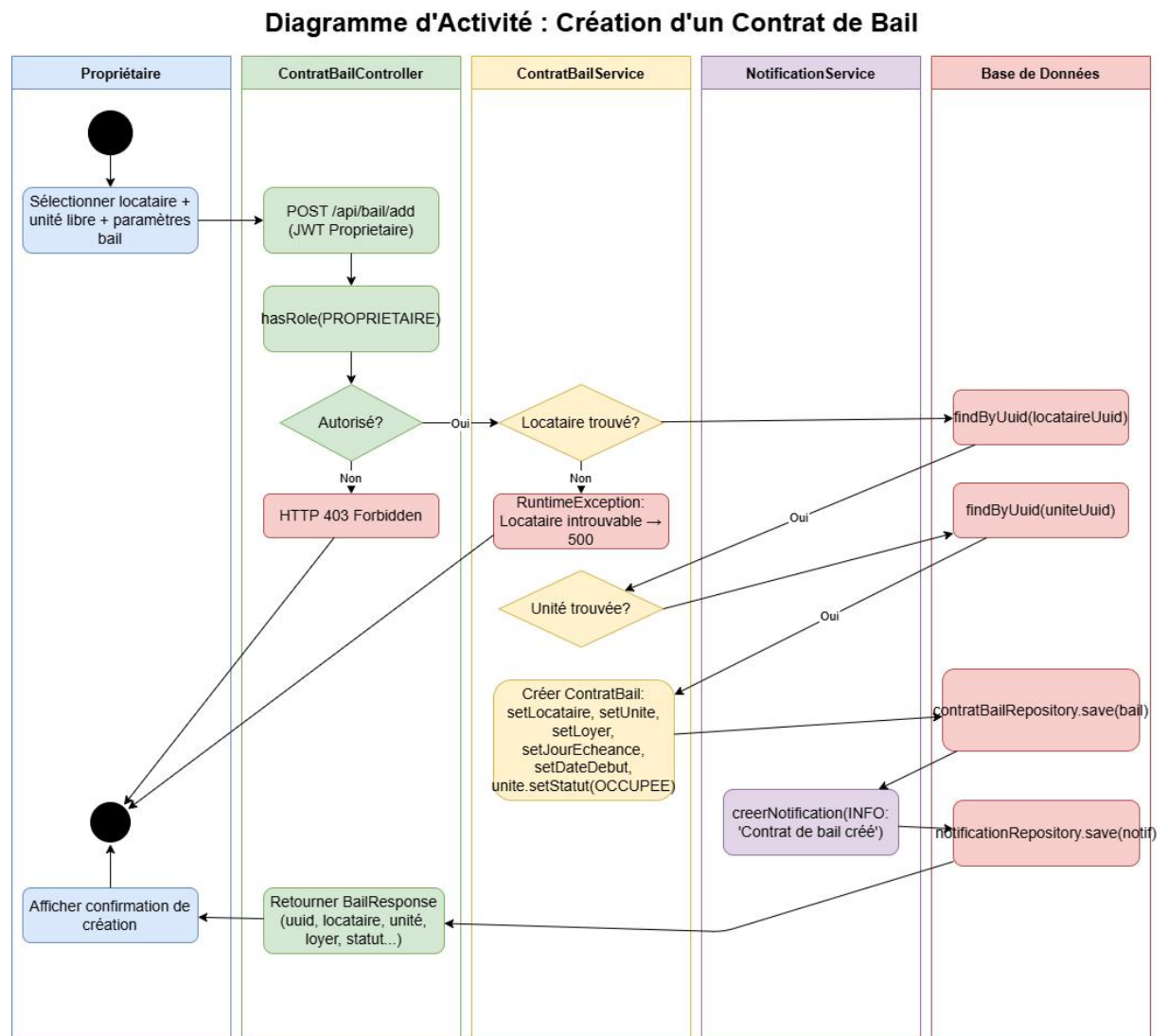


Figure 3.8 : Diagramme d'activité de création d'un bail

4. Le diagramme ci-dessous présente le processus de génération des avis d'échéances.

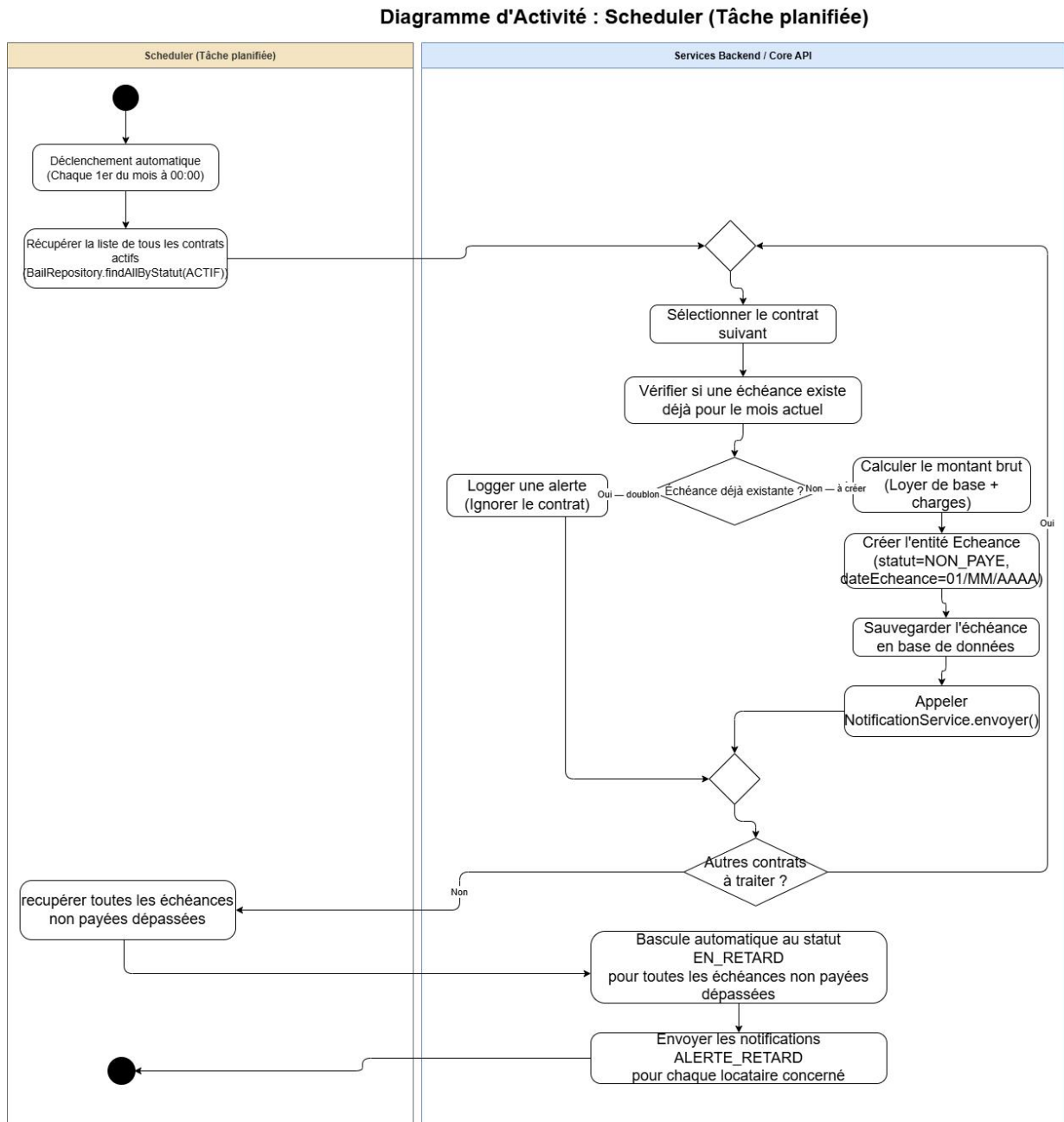


Figure 3.9 : Diagramme d'activit  du scheduler

5. Le diagramme ci-dessous présente le processus de paiement mobile.

Diagramme d'Activité : Paiement Mobile (FedaPay)

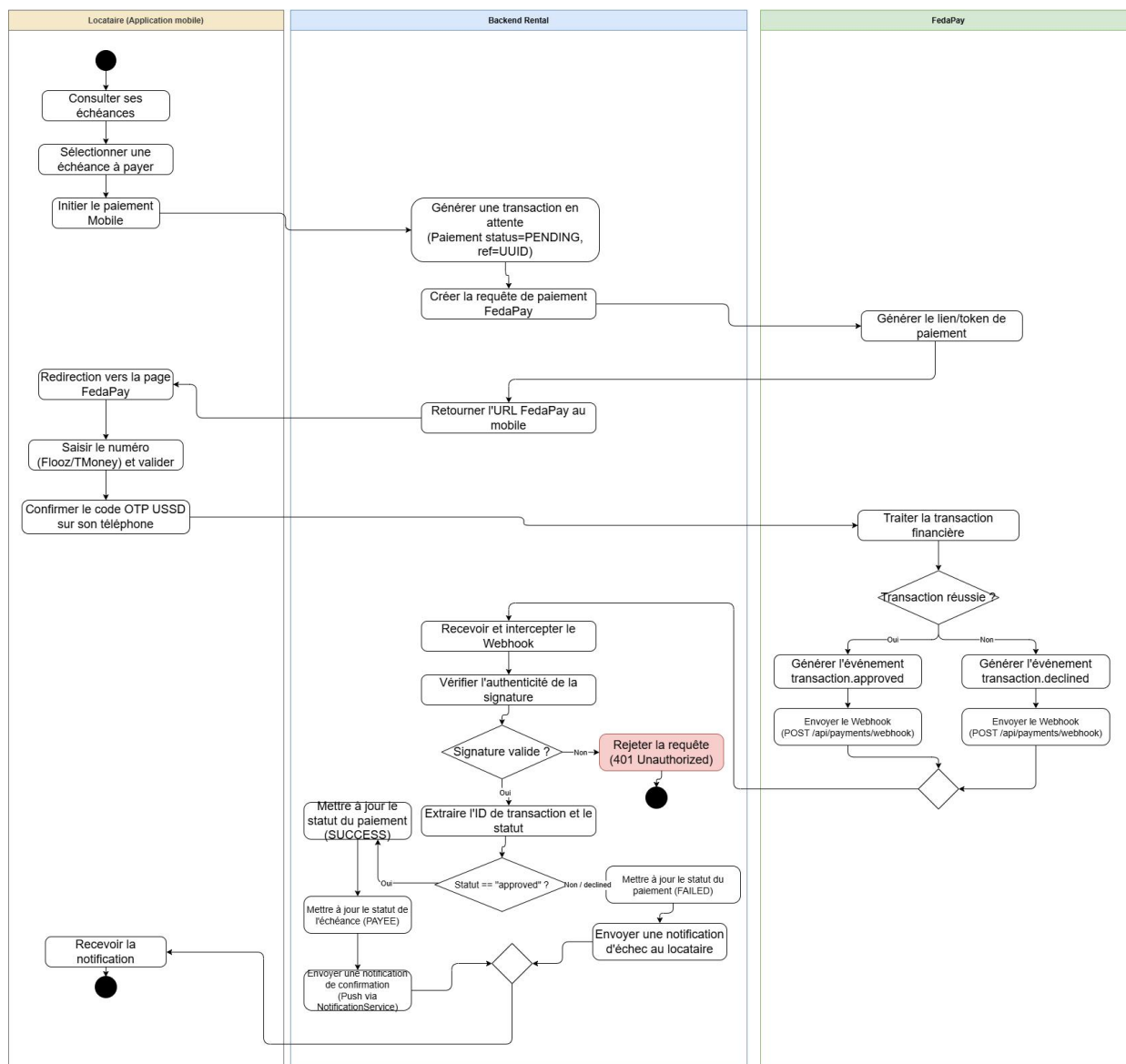


Figure 3.10 : Diagramme d'activité du paiement par mobile Money

3.4.5. Diagrammes de séquences

Les diagrammes de séquences décrivent les interactions chronologiques entre les objets du système pour un scénario donné. Ils permettent de visualiser précisément les appels entre le client (web ou mobile), le contrôleur REST, la couche service et la base de données, ainsi que les échanges avec FedaPay pour le paiement.

1. Séquence de création d'un bail

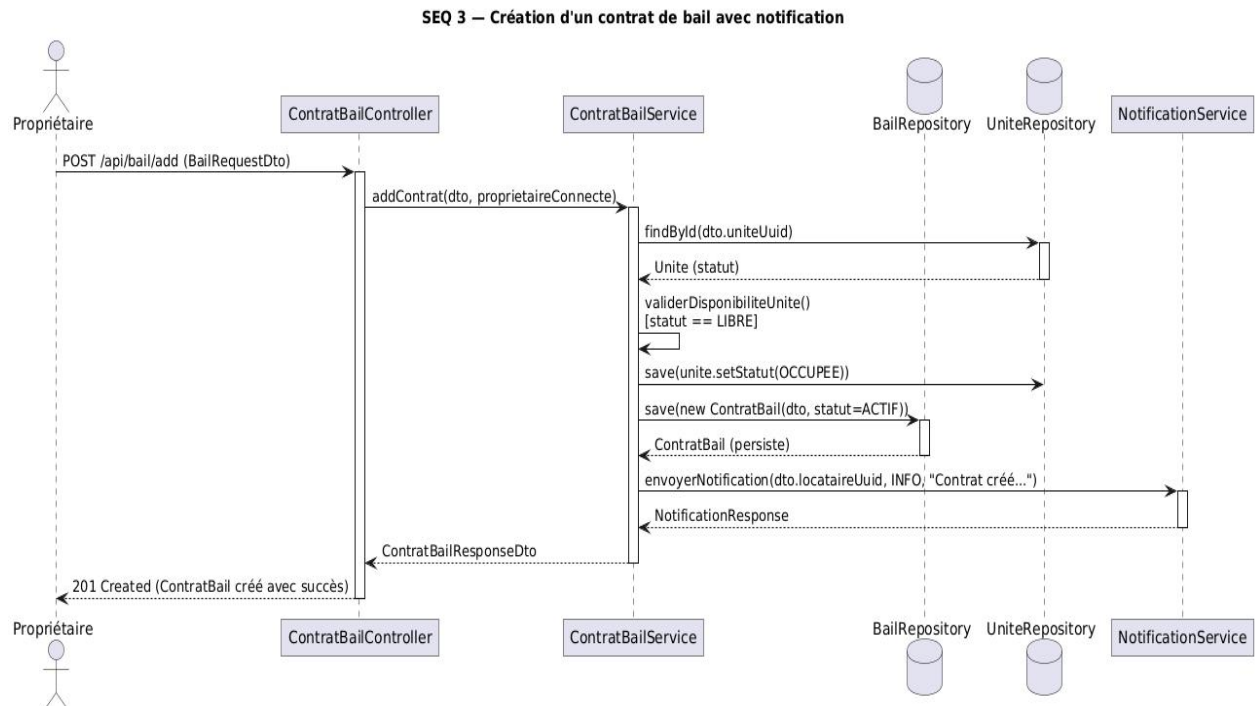


Figure 3.11 : Diagramme de séquence: création d'un bail

2. Séquence de génération mensuelle des échéances

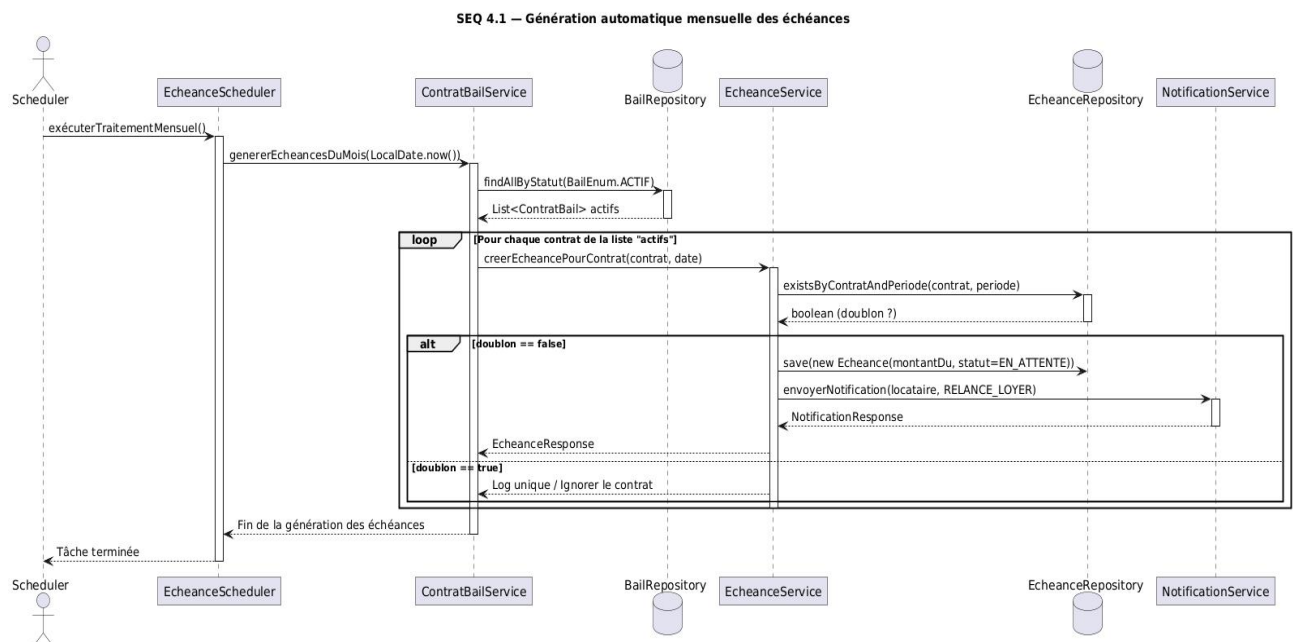


Figure 3.12 : Diagramme de séquence pour la Génération automatique des échéances

3. Séquence de vérification et de marquage automatique des retards d'échéances

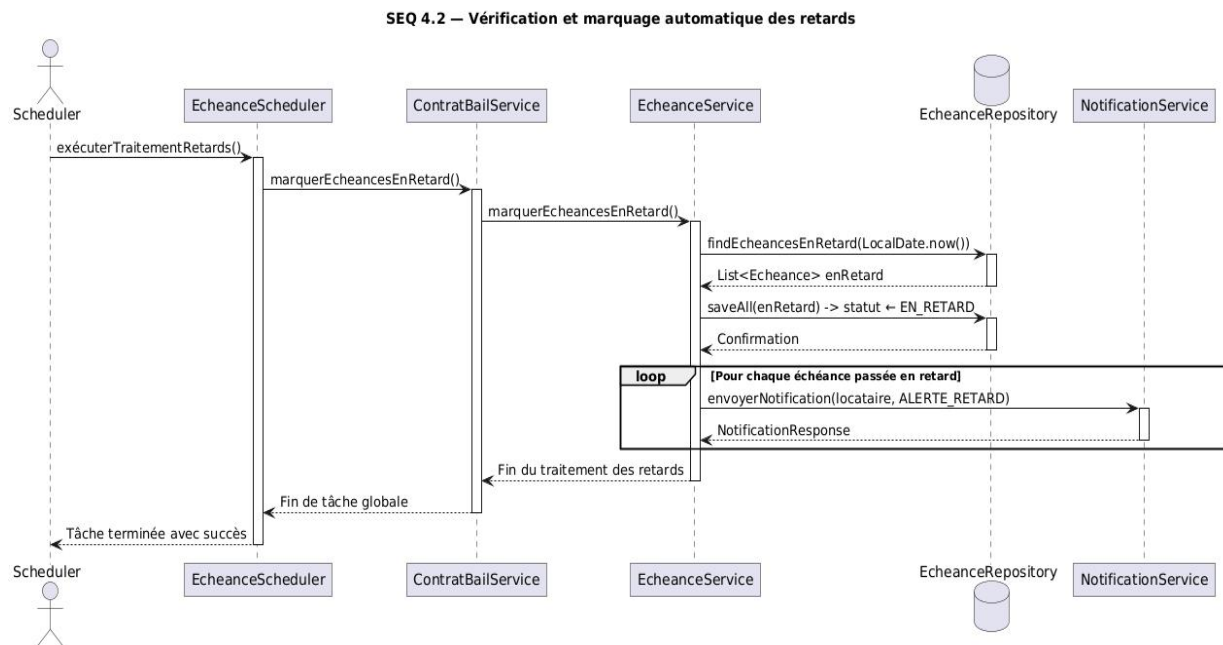


Figure 3.13 : Diagramme de séquence: vérification et le marquage des retards d'échéances

4. Séquence de paiement mobile

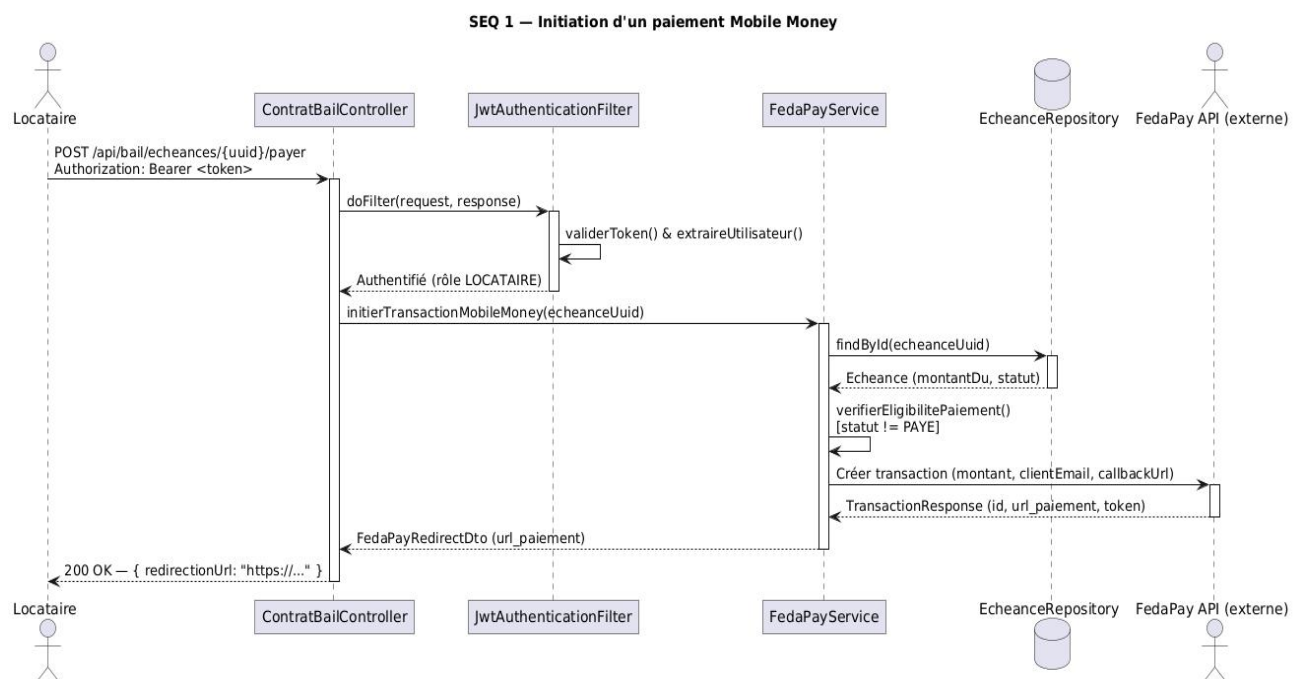


Figure 3.14 : Diagramme de séquence pour le paiement mobile money

5. Séquence de traitement du webhook de fedapay

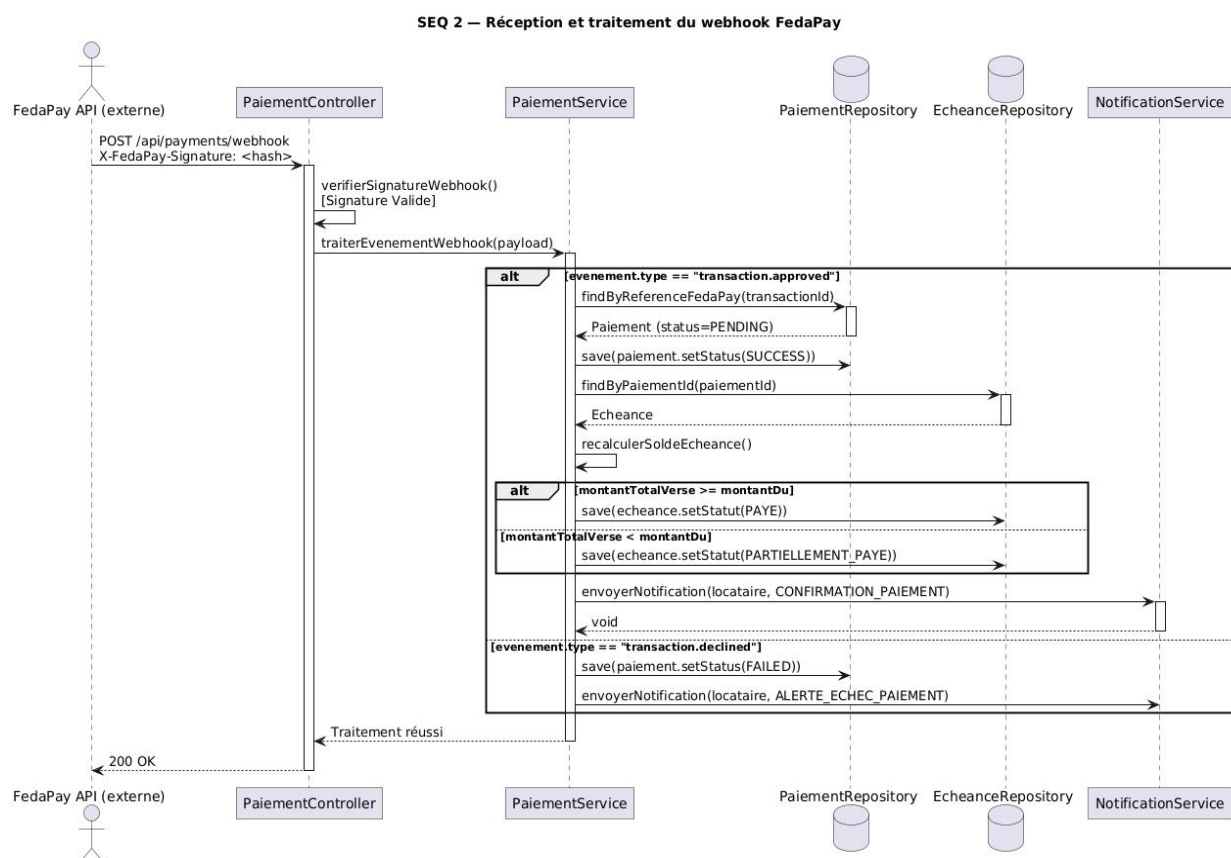


Figure 3.15 : Diagramme de séquence: réception et traitement d'un webhook fedapay

3.4.6. Diagramme de classes

Le diagramme de classes constitue la pierre angulaire de la conception orientée objet du projet. Il présente les entités métier du système (Utilisateur, Bien, Unite, ContratBail, Echeance, Paiement, Notification, Reservation), leurs attributs, leurs méthodes et les relations qui les lient (associations, héritage, composition). Ce modèle est directement mappé sur le schéma relationnel de la base de données PostgreSQL via l'ORM Hibernate/JPA.

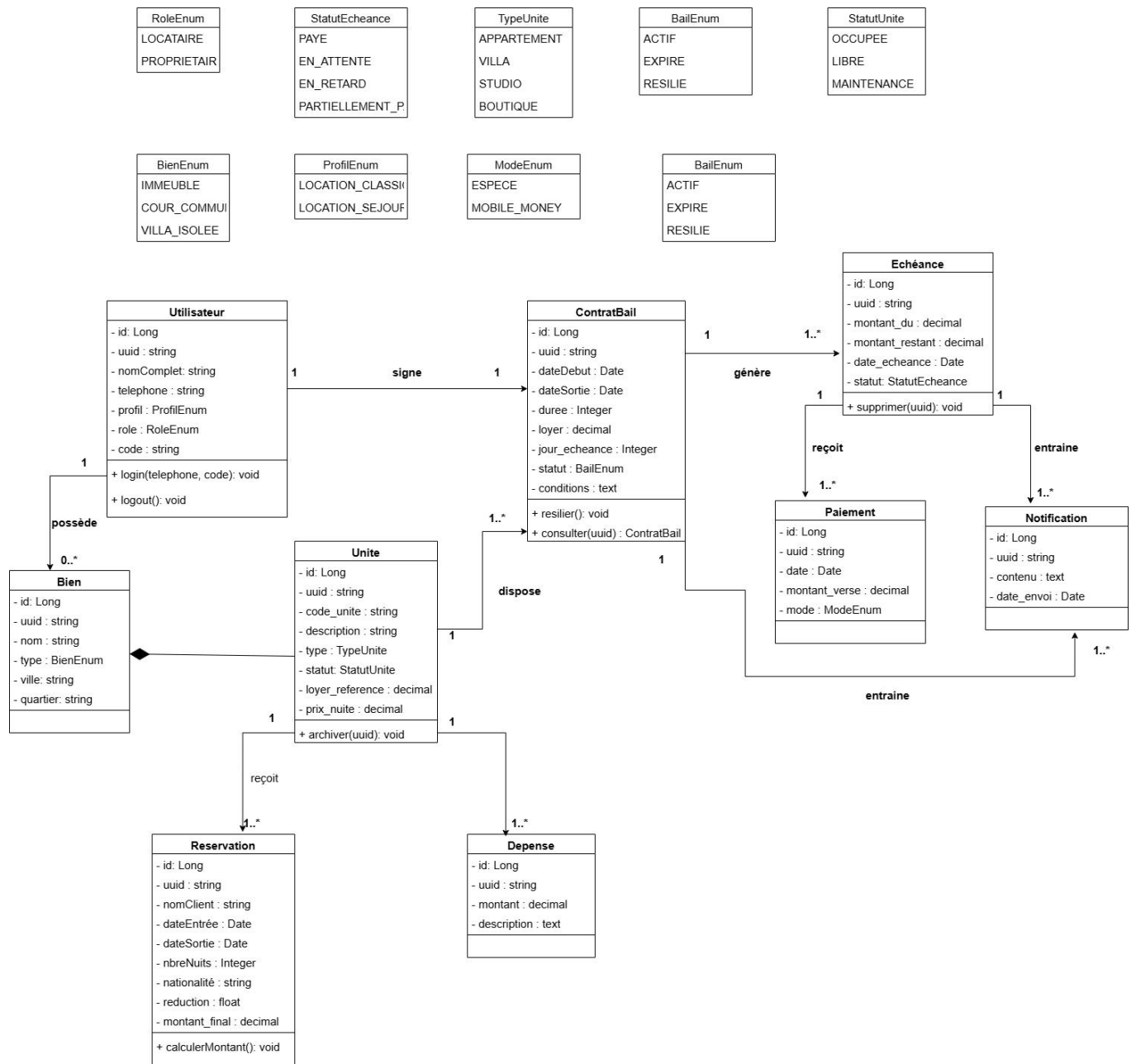


Figure 3.16 : Diagramme de classes du système

3.4.7. Diagrammes d'objets

Le diagramme d'objets illustre le système à un moment précis T. Il représente des instances réelles des classes avec des valeurs d'attributs spécifiques, permettant de valider la cohérence du diagramme de classes sur un scénario d'utilisation réel

L'exemple ci dessous nous montre un contrat de bail actif liant un propriétaire, un locataire et une unité précise etc...

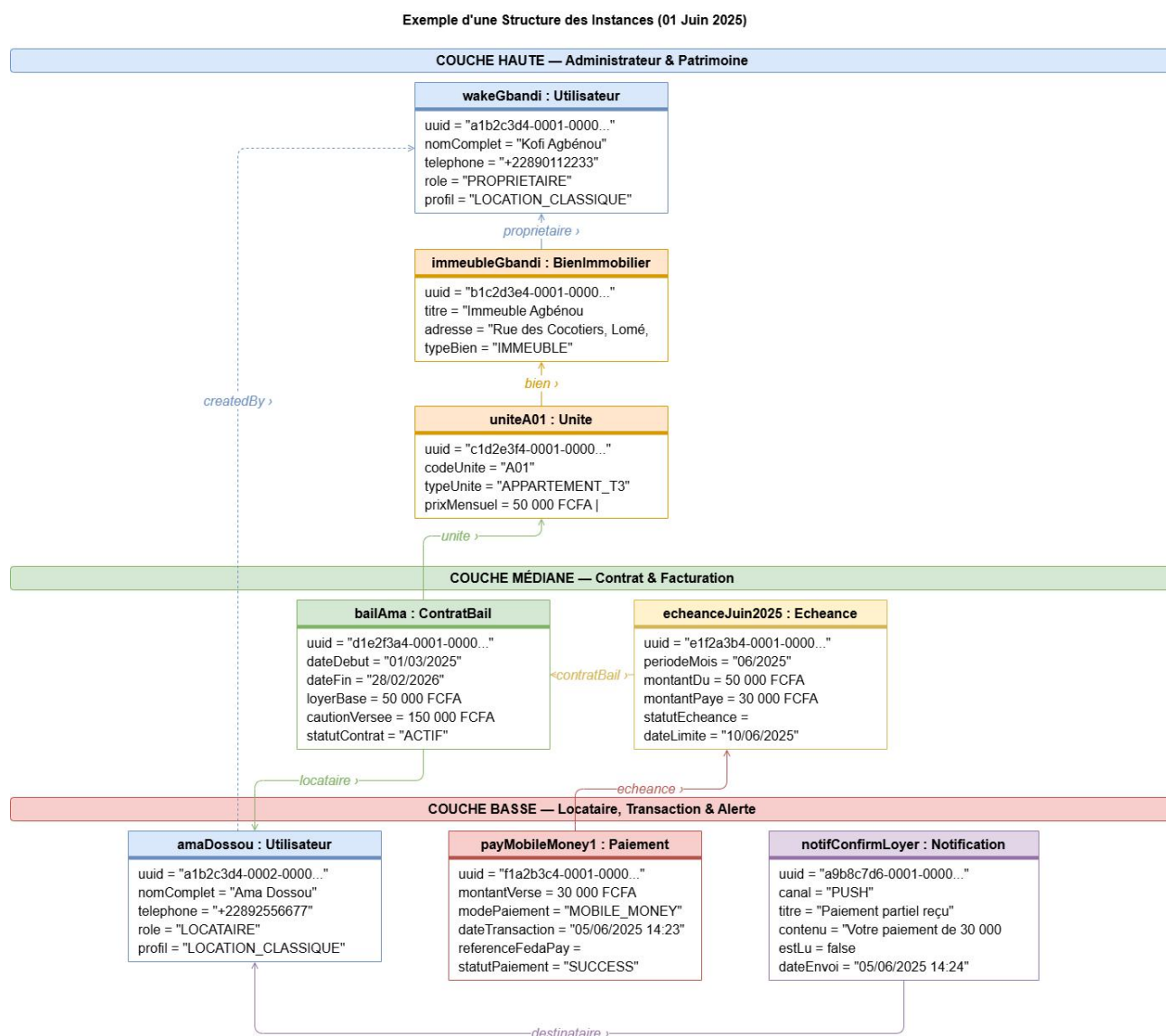


Figure 3.17 : Diagramme d'objet

3.4.8. Diagramme d'état-transition

Le diagramme d'état-transition modélise le cycle de vie des entités clés dont le comportement évolue dans le temps. En particulier, l'entité Echéance transite entre les états : NON_PAYÉ → EN_RETARD → PAYÉ. L'entité ContratBail suit quant à elle le cycle : ACTIF → RÉSILIÉ. Ces transitions sont déclenchées par des événements système (paiement reçu, date dépassée, résiliation manuelle).

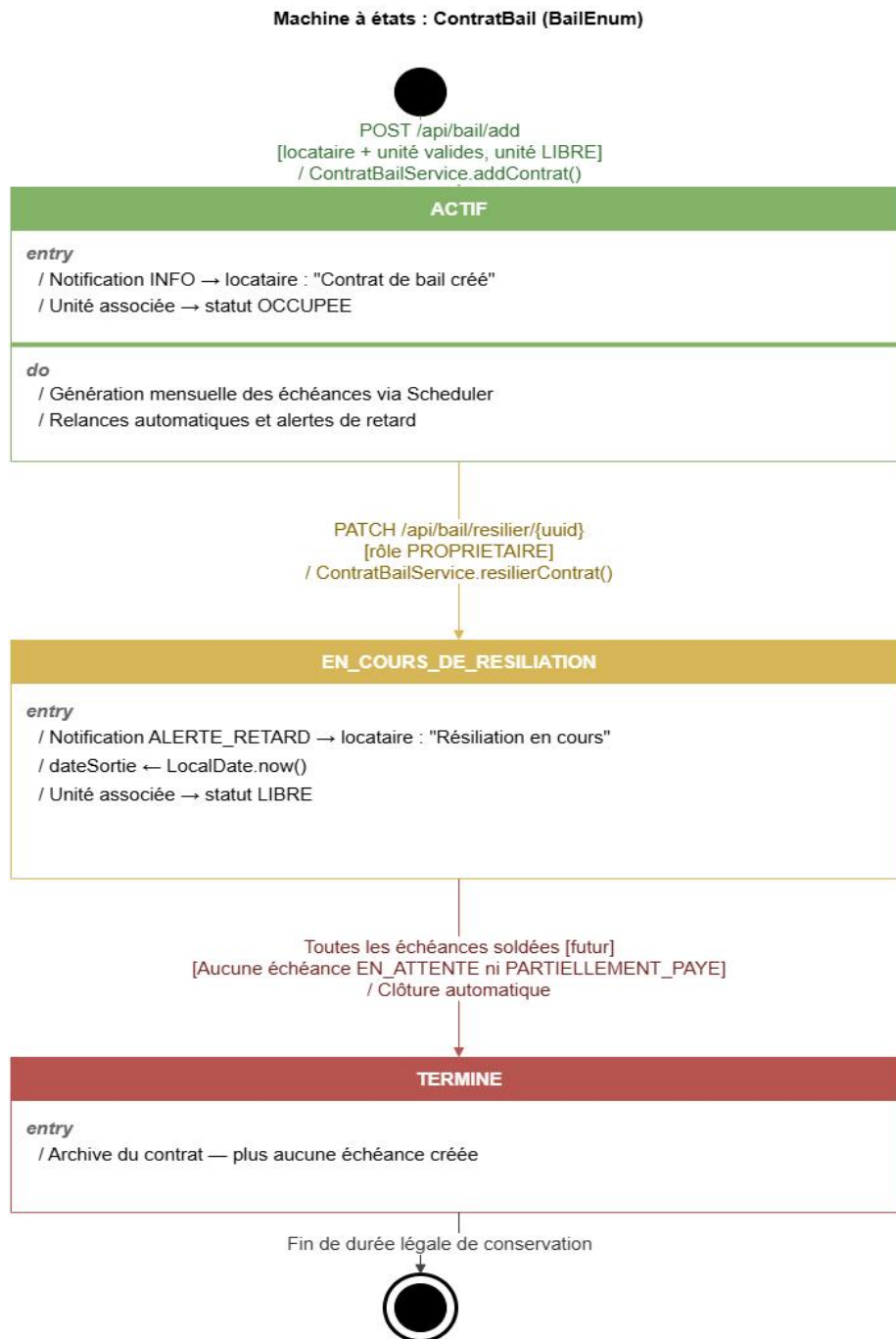


Figure 3.18 : Diagramme d'état-transition: Contrat de bail

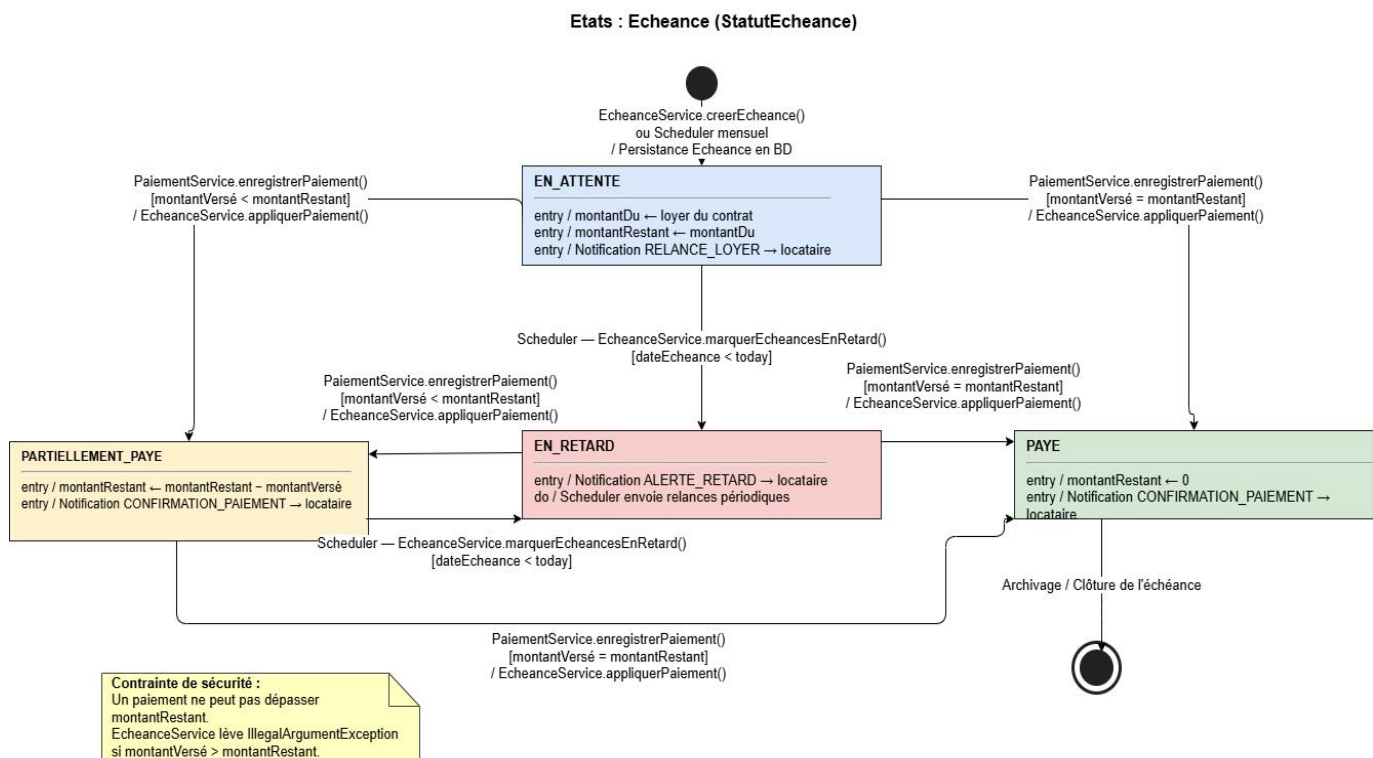


Figure 3.19 : Diagramme d'état-transition: Echéance

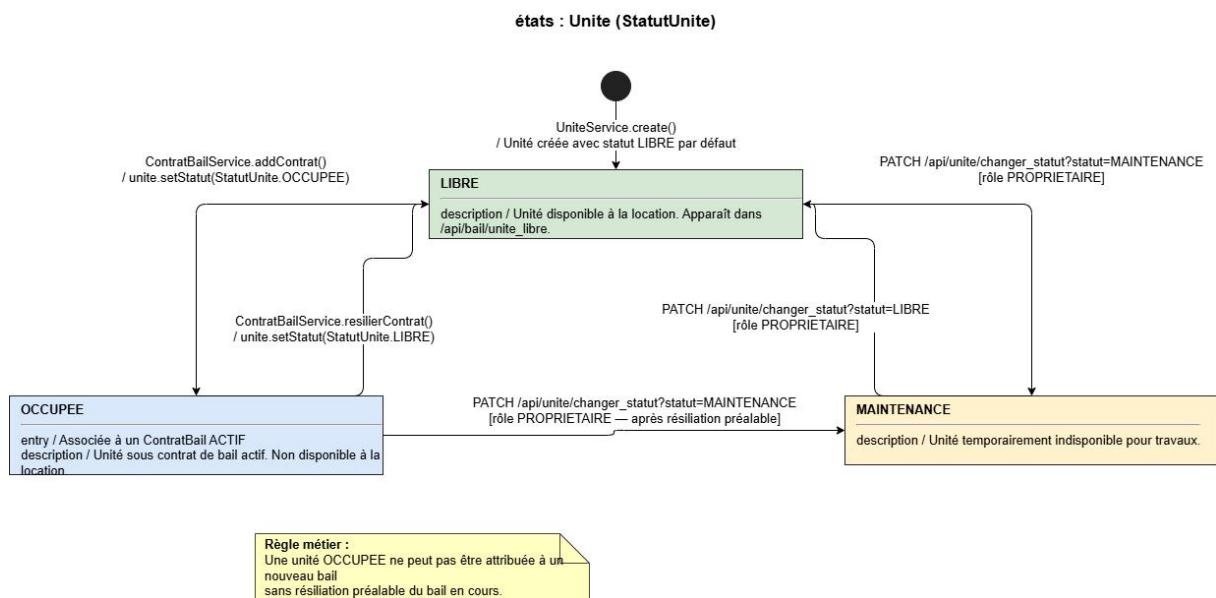


Figure 3.20 : Diagramme d'état-transition: Unité de logement

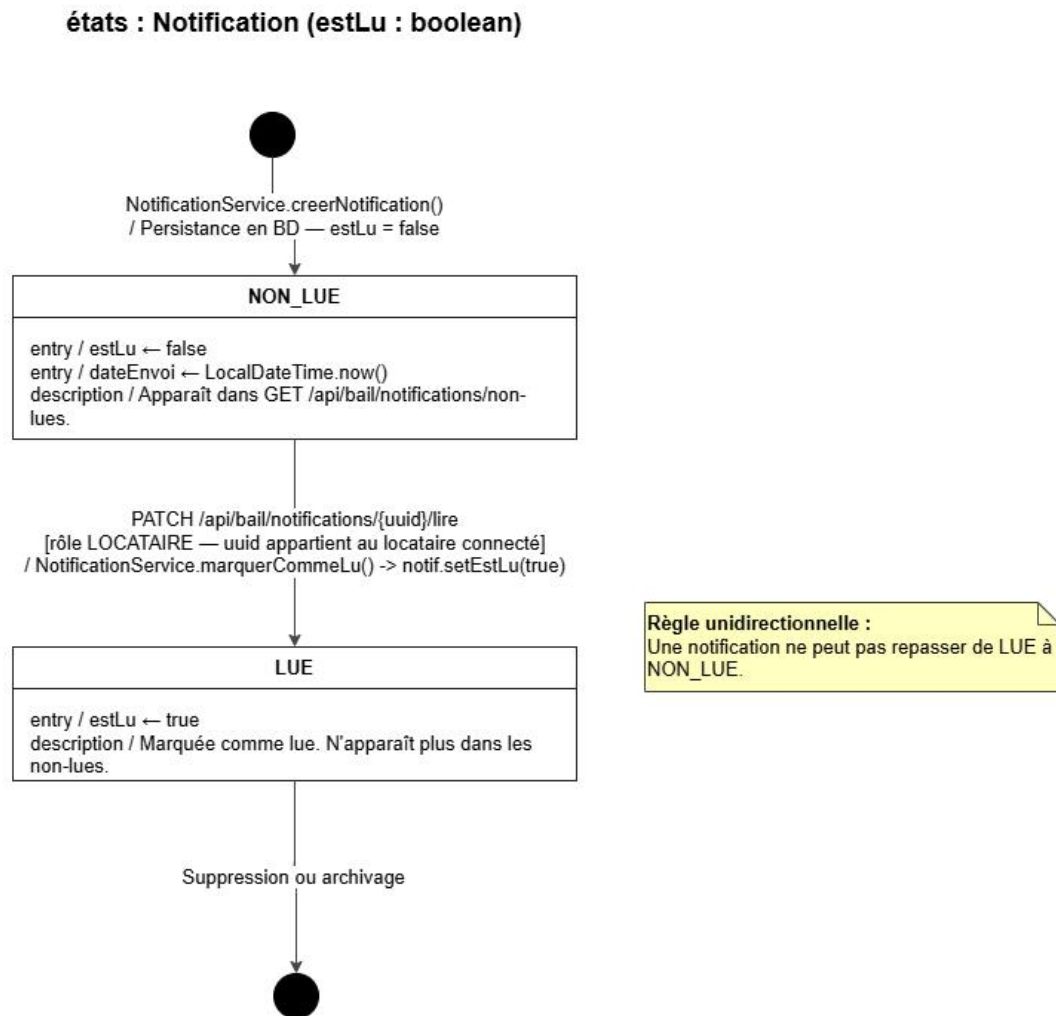


Figure 3.21 : Diagramme d'état-transition: Notification

3.4.9. Modèle Entité – Relationnel

Le modèle entité-relationnel (MER) représente la structure de la base de données PostgreSQL du projet. Il traduit le diagramme de classes en tables relationnelles, avec leurs clés primaires, clés étrangères et contraintes d'intégrité. Ce modèle a servi de base à la génération automatique du schéma via Hibernate (DDL auto).

Diagramme Entité-Relation

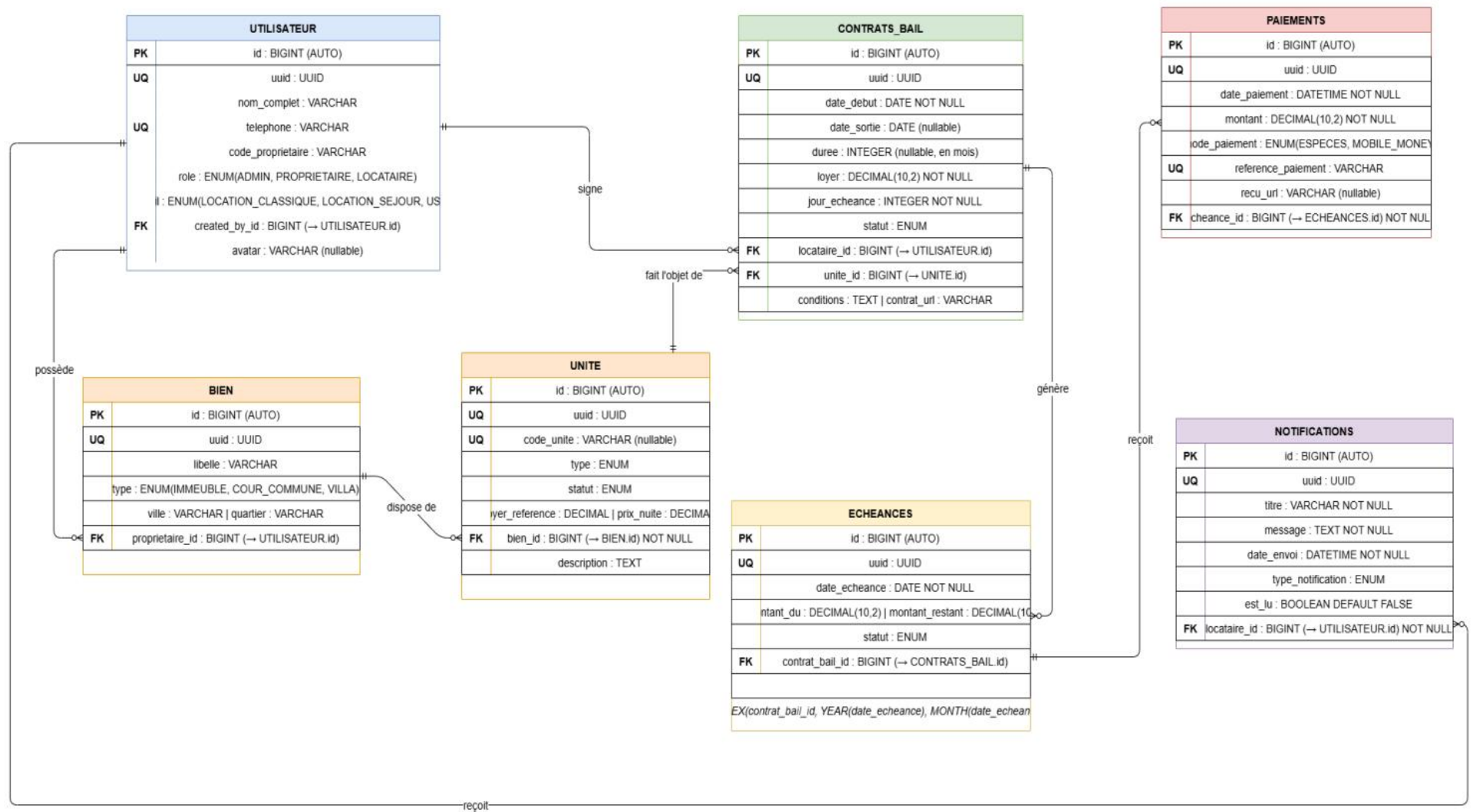


Figure 3.22 : Model entité relationnel

Chapitre 4 : REALISATION

4.1. Réalisation

4.1.1. Les matériels utilisés

La phase de réalisation et d'intégration de notre plateforme web et mobile a nécessité l'utilisation de ressources matérielles adaptées aux exigences de calcul. Les équipements retenus se composent de deux éléments principaux :

- L'environnement de développement (PC portable) : Le codage du backend Spring Boot, des interfaces Angular/Ionic, ainsi que la gestion de la base de données PostgreSQL ont été entièrement exécutés sur une machine aux caractéristiques suivantes :
 - ✓ Processeur : Intel Core i7.
 - ✓ Mémoire Vive (RAM) : 8 Go, indispensables pour faire tourner simultanément le serveur de développement Spring Boot, le compilateur Angular, l'instance de base de données et les outils de développement.
 - ✓ Stockage : Disque SSD de 236 Go, garantissant une bonne vitesse d'exécution et de compilation.
 - ✓ Système d'exploitation (OS) : Windows 10 Professionnel.
- Le terminal de test mobile (Smartphone Android) : Afin de valider l'ergonomie, la réactivité des notifications push et le bon fonctionnement du module de paiement Mobile Money local (Fedapay) en situation réelle, l'application mobile Ionic a été déployée et testée sur un smartphone physique :
 - ✓ Appareil : Smartphone sous environnement Android (version 10.0 ou supérieure, conforme à notre exigence non fonctionnelle d'exécuter au minimum Android 8.0).
 - ✓ Rôle : Validation de l'affichage adaptatif (responsive), des interactions tactiles de l'espace locataire et simulation des parcours de paiement par Flooz et T-Money.

4.1.2. Les logiciels utilisés

Le développement, le suivi de version et le déploiement de la plateforme ont reposé sur un ensemble d'outils logiciels et d'environnements de développement intégrés (IDE) adaptés aux standards actuels du Génie Logiciel :

- Spring Tool Suite : Choisi comme IDE principal pour le développement du backend en Java avec Spring Boot. Sa gestion native de Maven, la complétion avancée et ses outils de refactorisation ont facilité la mise en place de l'architecture en couches (Controller, Service, Repository).

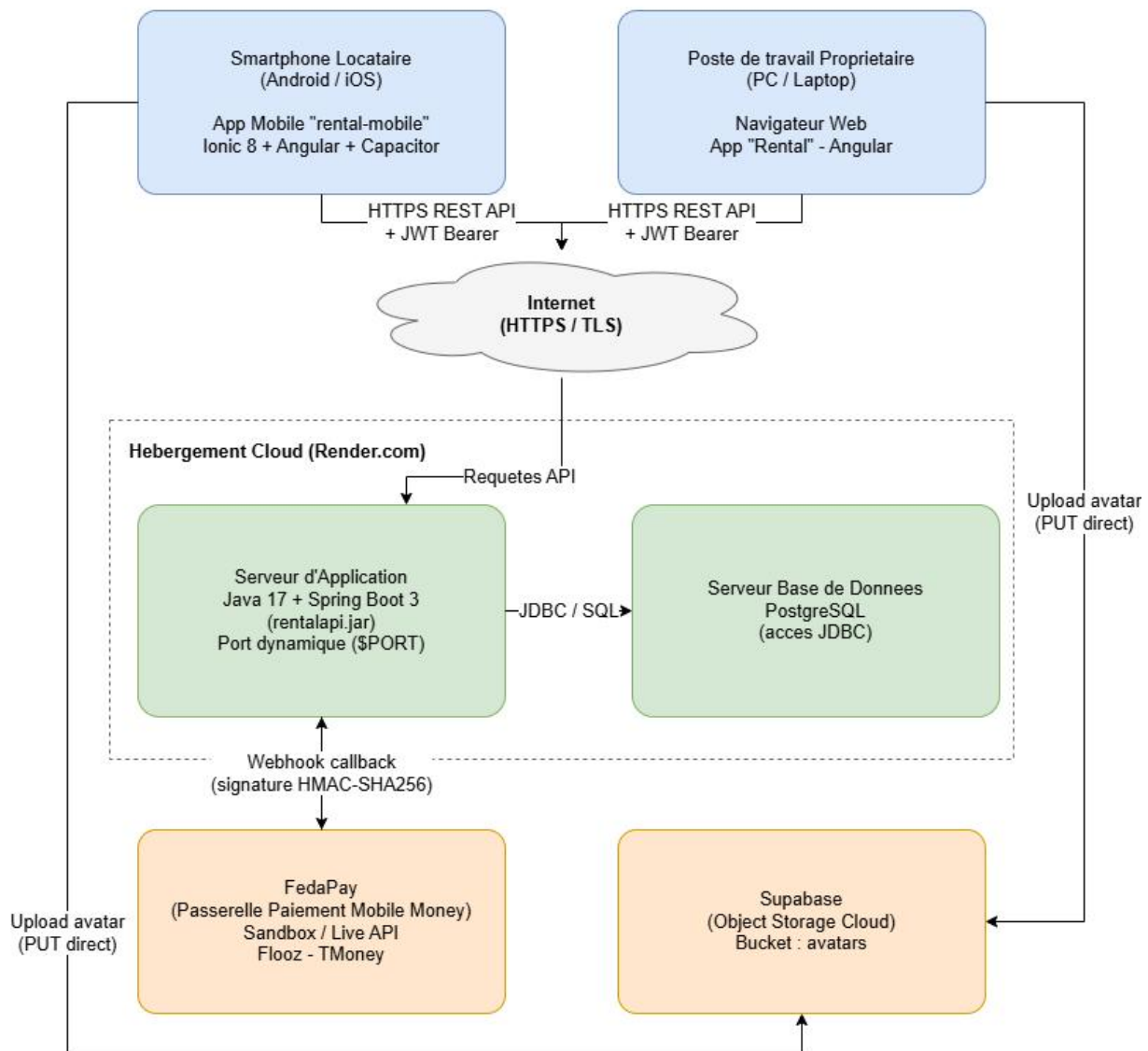
- Web Storm : Utilisé pour le développement des interfaces frontend Web (Angular) et Mobile (Ionic/Angular). Sa légèreté et son riche écosystème (TypeScript, Angular Snippets) ont offert un confort de codage optimal.
- Android Studio & Navigateurs Web : Android Studio a servi à configurer le SDK Android et à générer les builds d'émulation pour les tests mobiles. En phase de développement rapide, les navigateurs web (Google Chrome / Mozilla Firefox) ont été exploités via l'outil de débogage *Ionic Serve* pour tester le rendu de l'application en temps réel.
- PostgreSQL (avec pgAdmin)/Supabase : Système de gestion de base de données relationnelle (SGBDR) retenu pour stocker et sécuriser l'intégralité des données du système (biens, contrats, paiements).
- Postman : Outil indispensable pour tester, documenter et valider le comportement de nos API REST Spring Boot (notamment pour vérifier la génération et la validité des jetons de sécurité JWT) avant leur intégration dans le frontend.
- Draw.io / PlantUML : Outils logiciels dédiés à la modélisation visuelle et technique de l'ensemble des diagrammes UML 2.x et du modèle relationnel (ERD) détaillés au chapitre précédent.
- Git & GitHub : Git a été utilisé pour le contrôle de version local du code source. GitHub a servi de dépôt distant sécurisé, facilitant la sauvegarde du code, le suivi des modifications et l'automatisation des flux de déploiement.
- Render : Plateforme Cloud PaaS (Platform as a Service) retenue pour l'hébergement et le déploiement en ligne de notre API backend Spring Boot.

4.1.3. Architectures matérielles et logicielles

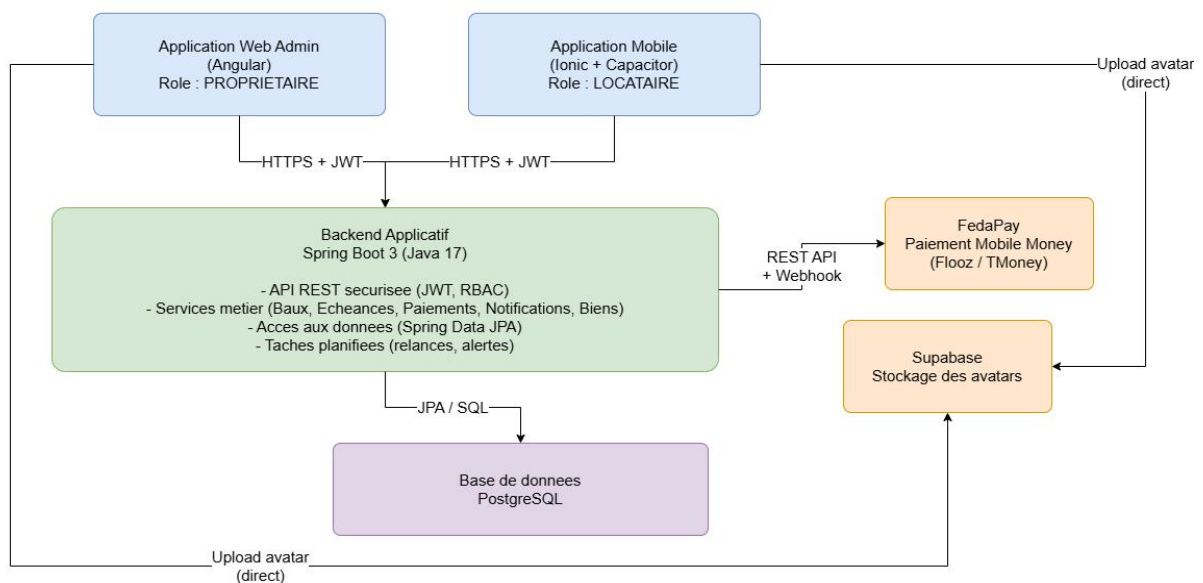
L'architecture retenue est une architecture **trois tiers** : le **tier présentation** (Angular web + Ionic mobile), le **tier métier** (API REST Spring Boot hébergé sur Render), et le **tier données** (PostgreSQL hébergé sur Supabase). Les communications entre tiers sont sécurisées par JWT en HTTPS. Le webhook FedaPay communique directement avec le backend via l'endpoint `/api/payments/webhook`, avec vérification de signature HMAC-SHA256.

Figure 4.1 : Architecture Logicielle et Matérielle de la plateforme

Architecture Materielle - Plateforme



Architecture Logicielle - Plateforme



4.1.4. Sécurité des applications

La nature critique des données manipulées par notre plateforme (informations personnelles des locataires, contrats de bail, flux financiers et transactions) impose la mise en œuvre de mécanismes de protection adaptée. Pour répondre à notre exigence non fonctionnelle d'isolation et de protection contre les attaques, la sécurité du système s'articule autour de quatre axes majeurs :

1. Authentification centralisée et gestion des jetons JWT

Afin de sécuriser les API REST du backend sans maintenir de session sur le serveur (*stateless*), nous avons implémenté une authentification basée sur les jetons JWT (JSON Web Token). Le mécanisme repose sur un double système de jetons pour garantir à la fois la sécurité et une expérience utilisateur fluide :

- Access Token (Durée de validité : 15 minutes) : Signé numériquement par le backend, ce jeton éphémère accompagne chaque requête HTTP dans l'en-tête *Authorization* pour authentifier l'utilisateur de manière sécurisée.
- Refresh Token (Durée de validité : 7 jours) : Stocké de manière sécurisée côté client, ce jeton à longue durée permet à l'application Angular ou Ionic de solliciter automatiquement un nouvel *Access Token* sans forcer l'utilisateur à ressaisir ses identifiants, réduisant ainsi l'exposition du jeton d'accès en cas d'interception.

2. Contrôle d'accès par rôle avec Spring Security

Le framework Spring Security orchestre la sécurité du backend en interceptant chaque requête à travers une chaîne de filtres personnalisés. L'accès aux ressources est strictement cloisonné selon les rôles définis dans notre modèle relationnel (PROPRIETAIRE, LOCATAIRE). Au niveau du code source (couche Controller), nous utilisons l'annotation `@PreAuthorize` pour appliquer un contrôle d'accès granulé basé sur les expressions de sécurité de Spring. Par exemple :

- `@PreAuthorize("hasRole('PROPRIETAIRE')")` restreint l'accès aux modules de création de baux ou d'édition de biens (Profils A et B).
- `@PreAuthorize("hasRole('LOCATAIRE')")` autorise l'accès en lecture seule de l'historique des factures de l'application mobile.

3. Isolation stricte des données par propriétaire

Dans un environnement où plusieurs propriétaires gèrent leurs patrimoines respectifs sur la même base de données PostgreSQL, l'isolation logique des données est obligatoire. Lors de chaque requête de lecture, de modification ou de suppression, la couche *Service* de Spring Boot injecte dynamiquement l'identifiant du propriétaire actuellement authentifié (extrait de l'Access Token JWT). Toutes les requêtes SQL générées par Hibernate intègrent une clause de filtrage (ex: `WHERE bien.proprietaire_id = :current_user_id`), interdisant formellement à un utilisateur de visualiser ou de manipuler les données d'un autre bailleur.

4. Validation HMAC des webhooks FedaPay

Le règlement des loyers via Mobile Money s'effectuant de manière asynchrone, la passerelle FedaPay notifie notre backend de la réussite d'un paiement en lui envoyant une requête HTTP de type Webhook. Pour empêcher des attaquants de simuler de fausses confirmations de paiement, le backend réalise une validation de signature HMAC (Hash-based Message Authentication Code). Lors de la réception de la notification, Spring Boot intercepte la signature contenue dans l'en-tête de la requête et la recalcule localement en combinant le corps de la requête avec la clé secrète partagée de FedaPay. Le statut de la facture n'est mis à jour à l'état "Payé" que si les deux signatures correspondent à 100 %, garantissant ainsi l'intégrité absolue de nos transactions financières.

4.2. Scripts de création de la base de données

```
CREATE TABLE utilisateurs (  
    id          BIGSERIAL PRIMARY KEY,  
    uuid        UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),  
    nom_complet VARCHAR(150) NOT NULL,  
    telephone   VARCHAR(20) NOT NULL UNIQUE,  
    code_proprietaire VARCHAR(255) NOT NULL,          -- code d'accès / mot de passe (haché)  
    profil      VARCHAR(30)  
                CHECK (profil IN ('LOCATION_CLASSIQUE', 'LOCATION_SEJOUR')),  
    role        VARCHAR(20) NOT NULL  
                CHECK (role IN ('LOCATAIRE', 'PROPRIETAIRE'))  
);  
  
CREATE TABLE biens (  
    id          BIGSERIAL PRIMARY KEY,  
    uuid        UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),  
    nom         VARCHAR(150) NOT NULL,  
    type        VARCHAR(30) NOT NULL  
                CHECK (type IN ('IMMEUBLE', 'COUR_COMMUNE', 'VILLA_ISOLEE')),  
    ville       VARCHAR(100) NOT NULL,  
    quartier    VARCHAR(100),  
    proprietaire_id BIGINT NOT NULL REFERENCES utilisateurs(id) ON DELETE CASCADE  
);
```

```

CREATE INDEX idx_biens_proprietaire ON biens(proprietaire_id);

CREATE TABLE unites (

    id          BIGSERIAL PRIMARY KEY,

    uuid        UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),

    code_unite   VARCHAR(50) UNIQUE,

    description  TEXT,

    type         VARCHAR(30) NOT NULL

                CHECK (type IN ('APPARTEMENT', 'VILLA', 'STUDIO', 'BOUTIQUE')),

    statut       VARCHAR(30) NOT NULL DEFAULT 'LIBRE'

                CHECK (statut IN ('OCCUPEE', 'LIBRE', 'MAINTENANCE')),

    loyer_reference NUMERIC(12,2),

    prix_nuite    NUMERIC(12,2),

    bien_id       BIGINT NOT NULL REFERENCES biens(id) ON DELETE CASCADE

);

CREATE INDEX idx_unites_bien ON unites(bien_id);

CREATE TABLE contrats_bail (

    id          BIGSERIAL PRIMARY KEY,

    uuid        UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),

    date_debut   DATE NOT NULL,

    date_sortie  DATE,

    duree        INTEGER,

    loyer         NUMERIC(12,2) NOT NULL,

    jour_echeance INTEGER NOT NULL CHECK (jour_echeance BETWEEN 1 AND 31),

    statut       VARCHAR(20) NOT NULL DEFAULT 'ACTIF'

                CHECK (statut IN ('ACTIF', 'EXPIRE', 'RESILIE')),

    conditions   TEXT,

    locataire_id BIGINT NOT NULL REFERENCES utilisateurs(id),

    unite_id     BIGINT NOT NULL REFERENCES unites(id),

    CONSTRAINT chk_contrat_dates CHECK (date_sortie IS NULL OR date_sortie > date_debut)

);

```

```

CREATE INDEX idx_contrats_locataire ON contrats_bail(locataire_id);

CREATE INDEX idx_contrats_unite ON contrats_bail(unite_id);

CREATE TABLE echeances (

    id          BIGSERIAL PRIMARY KEY,

    uuid        UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),

    montant_du   NUMERIC(12,2) NOT NULL,

    montant_restant NUMERIC(12,2) NOT NULL,

    date_echeance DATE NOT NULL,

    statut       VARCHAR(30) NOT NULL DEFAULT 'EN_ATTENTE'

    CHECK (statut IN ('PAYE', 'EN_ATTENTE', 'EN_RETARD', 'PARTIELLEMENT_PAYE')),

    contrat_bail_id BIGINT NOT NULL REFERENCES contrats_bail(id) ON DELETE CASCADE

);

CREATE INDEX idx_echeances_contrat ON echeances(contrat_bail_id);

CREATE TABLE paiements (

    id          BIGSERIAL PRIMARY KEY,

    uuid        UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),

    date_paiement TIMESTAMP NOT NULL DEFAULT now(),

    montant_verse NUMERIC(12,2) NOT NULL CHECK (montant_verse > 0),

    mode         VARCHAR(20) NOT NULL

    CHECK (mode IN ('ESPECE', 'MOBILE_MONEY')),

    echeance_id BIGINT NOT NULL REFERENCES echeances(id) ON DELETE CASCADE

);

CREATE INDEX idx_paiements_echeance ON paiements(echeance_id);

CREATE TABLE notifications (

    id          BIGSERIAL PRIMARY KEY,

    uuid        UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),

    contenu     TEXT NOT NULL,

    date_envoi  TIMESTAMP NOT NULL DEFAULT now(),

    echeance_id BIGINT REFERENCES echeances(id) ON DELETE CASCADE,

    paiement_id BIGINT REFERENCES paiements(id) ON DELETE CASCADE,

```

```

CONSTRAINT chk_notification_origine

CHECK (echeance_id IS NOT NULL OR paiement_id IS NOT NULL)

);

CREATE INDEX idx_notifications_echeance ON notifications(echeance_id);

CREATE INDEX idx_notifications_paiement ON notifications(paiement_id);

CREATE TABLE reservations (

    id          BIGSERIAL PRIMARY KEY,

    uuid        UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),

    nom_client   VARCHAR(150) NOT NULL,

    date_entree  DATE NOT NULL,

    date_sortie  DATE NOT NULL,

    nbre_nuits   INTEGER NOT NULL CHECK (nbre_nuits > 0),

    nationalite  VARCHAR(80),

    reduction    REAL DEFAULT 0 CHECK (reduction >= 0 AND reduction <= 100),

    montant_final NUMERIC(12,2),

    unite_id     BIGINT NOT NULL REFERENCES unites(id) ON DELETE CASCADE,

    CONSTRAINT chk_reservation_dates CHECK (date_sortie > date_entree)

);

CREATE INDEX idx_reservations_unite ON reservations(unite_id);

CREATE TABLE depenses (

    id          BIGSERIAL PRIMARY KEY,

    uuid        UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),

    montant     NUMERIC(12,2) NOT NULL CHECK (montant > 0),

    description  TEXT,

    unite_id     BIGINT NOT NULL REFERENCES unites(id) ON DELETE CASCADE

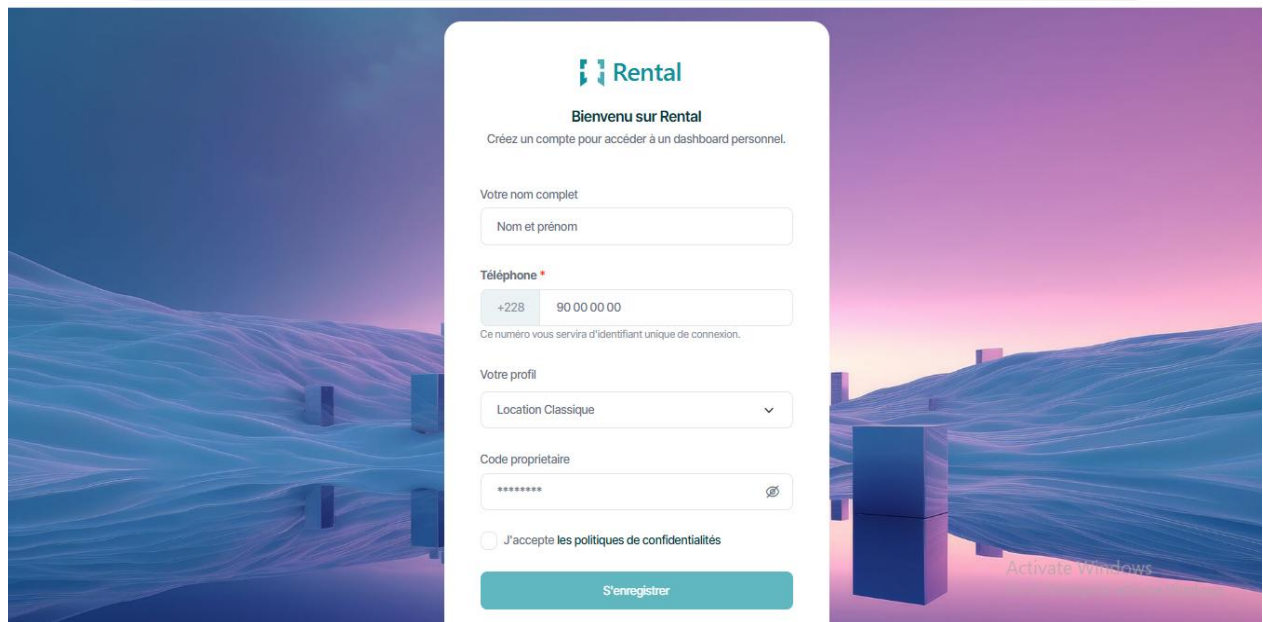
);

CREATE INDEX idx_depenses_unite ON depenses(unite_id);

```

4.3. Quelques captures des interfaces et codes sources des applications

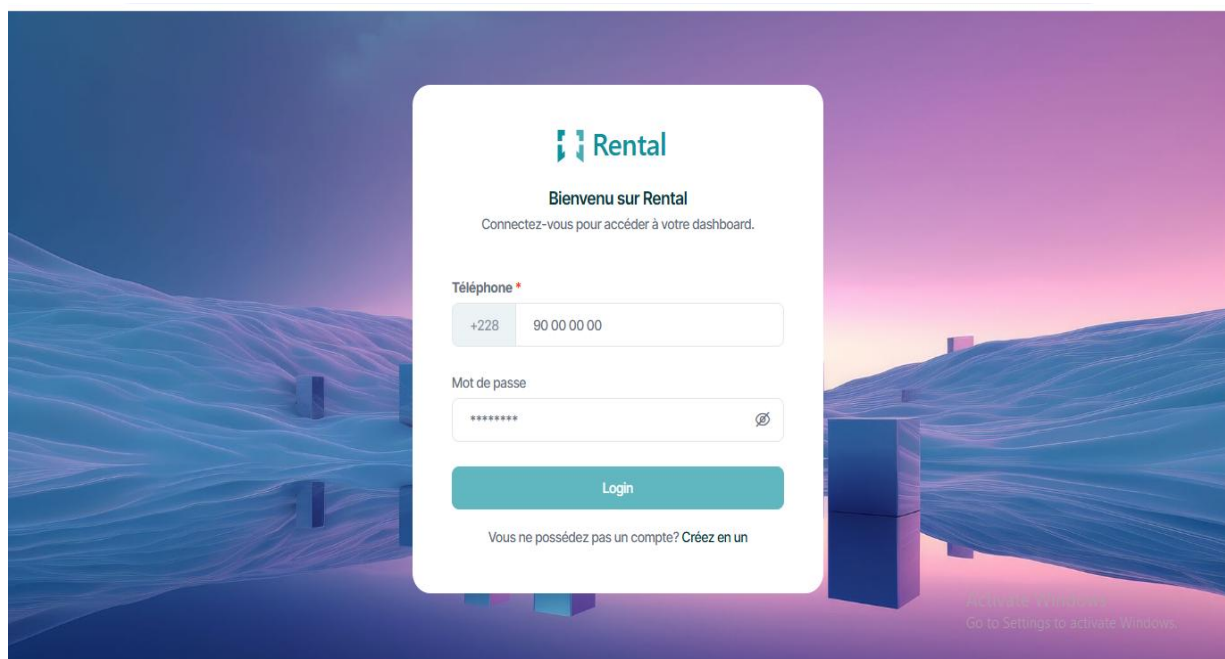
4.3.1. Captures des interfaces



The screenshot shows a registration form for 'Rental' on a website with a blue and purple abstract background. The form is titled 'Bienvenu sur Rental' and includes the following fields and elements:

- Header:** Rental logo and 'Bienvenu sur Rental'.
- Text:** 'Créez un compte pour accéder à un dashboard personnel.'
- Form Fields:**
 - 'Votre nom complet' with a placeholder 'Nom et prénom'.
 - 'Téléphone *' with a dropdown for '+228' and a text input for '90 00 00 00'.
 - 'Ce numéro vous servira d'identifiant unique de connexion.'
 - 'Votre profil' with a dropdown menu showing 'Location Classique'.
 - 'Code propriétaire' with a masked input '*****' and an eye icon.
 - A checkbox for 'J'accepte les politiques de confidentialités'.
- Button:** 'S'enregistrer'.
- Footer:** 'Active Windows' watermark.

Figure 4.3 : page web d'enregistrement



The screenshot shows a login form for 'Rental' on the same website. The form is titled 'Bienvenu sur Rental' and includes the following fields and elements:

- Header:** Rental logo and 'Bienvenu sur Rental'.
- Text:** 'Connectez-vous pour accéder à votre dashboard.'
- Form Fields:**
 - 'Téléphone *' with a dropdown for '+228' and a text input for '90 00 00 00'.
 - 'Mot de passe' with a masked input '*****' and an eye icon.
- Button:** 'Login'.
- Text:** 'Vous ne possédez pas un compte? Créez en un'.
- Footer:** 'Active Windows' watermark.

Figure 4.3 : page web de connection

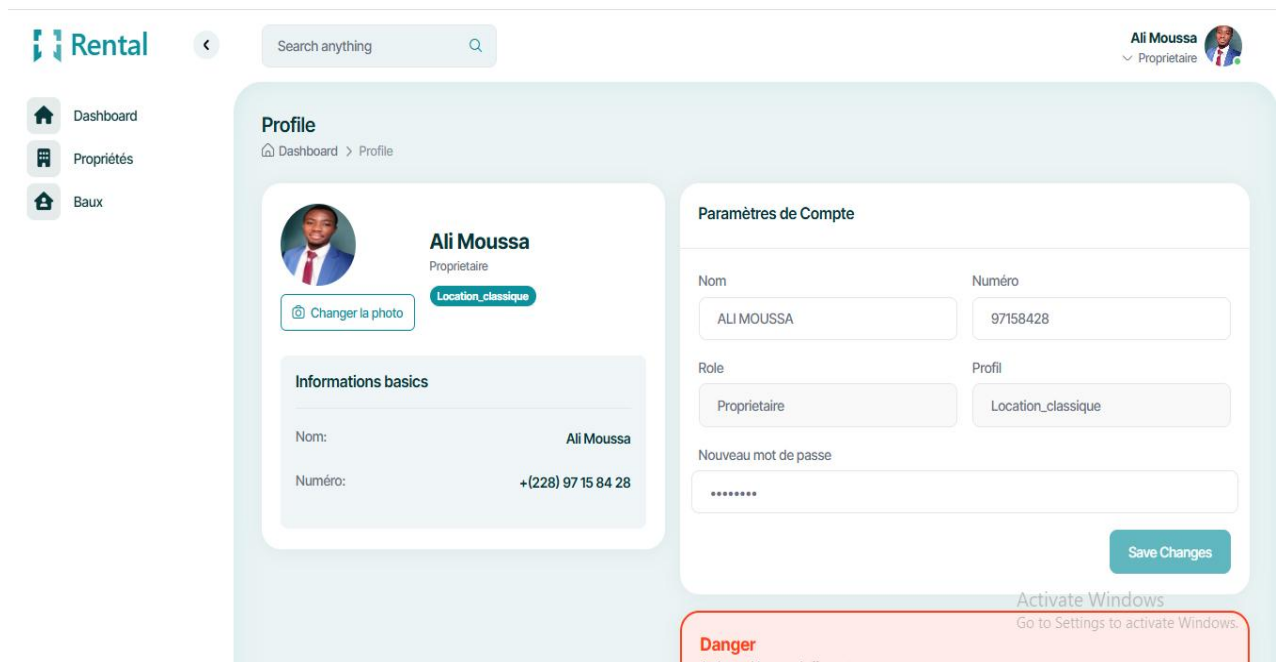


Figure 4.3 page web profil

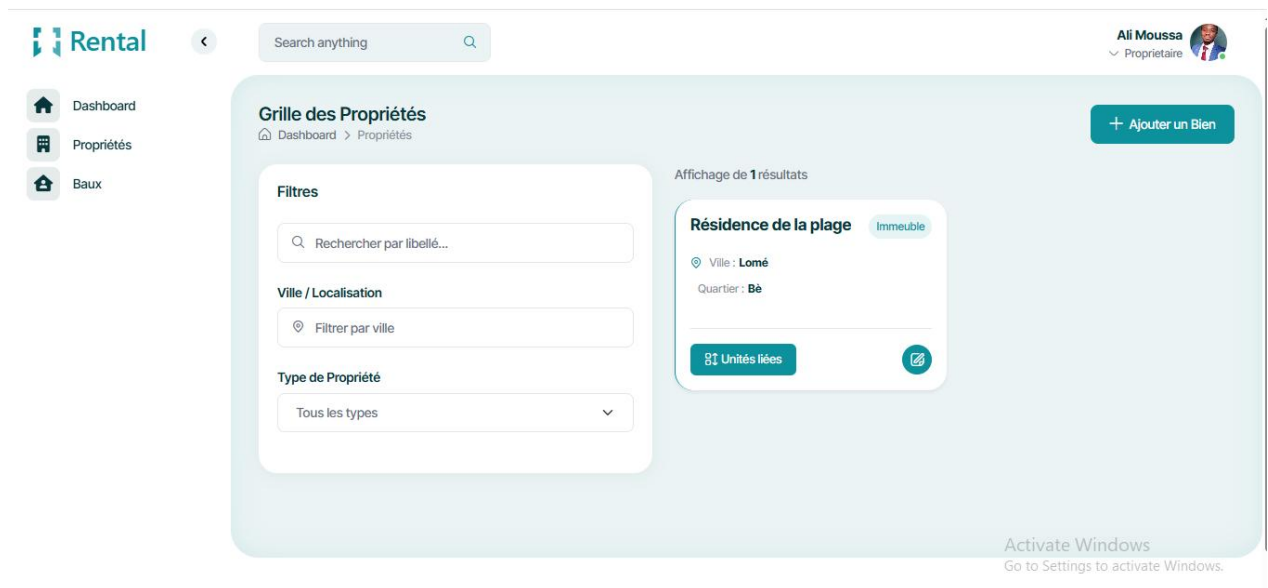


Figure 4.3 : page web des gestion des propriétés

Ajouter un nouveau bien

Libellé du bien
Ex: Résidence Amina

Type de bien
Immeuble

Ville
Ex: Lomé

Quartier
Ex: Adidogomé

Annuler Enregistrer

Figure 4.3 : formulaire d'ajout d'un nouveau bien

Ajouter une nouvelle unité

Code de l'unité
Ex: APT-B4

Type d'unité
Appartement

Loyer de référence (Mensuel)
Montant en F CFA

Statut initial
Libre

Description / Notes
Caractéristiques spécifiques...

Annuler Enregistrer

Figure 4.3 : formulaire d'ajout d'une unité de logement

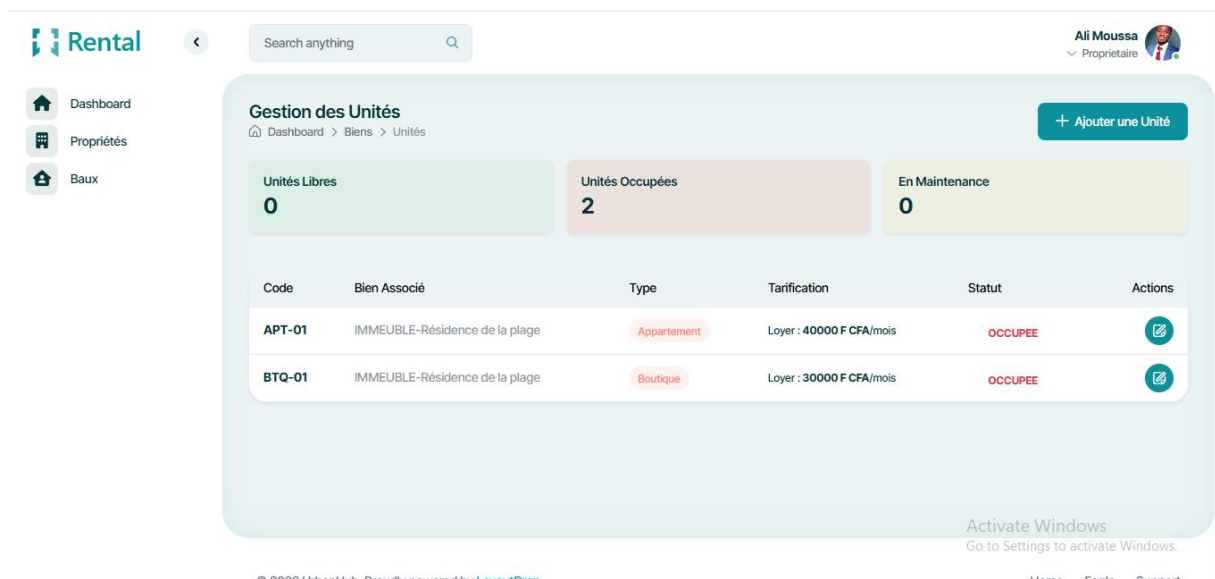


Figure 4.3 : page web de gestion des unités de logements

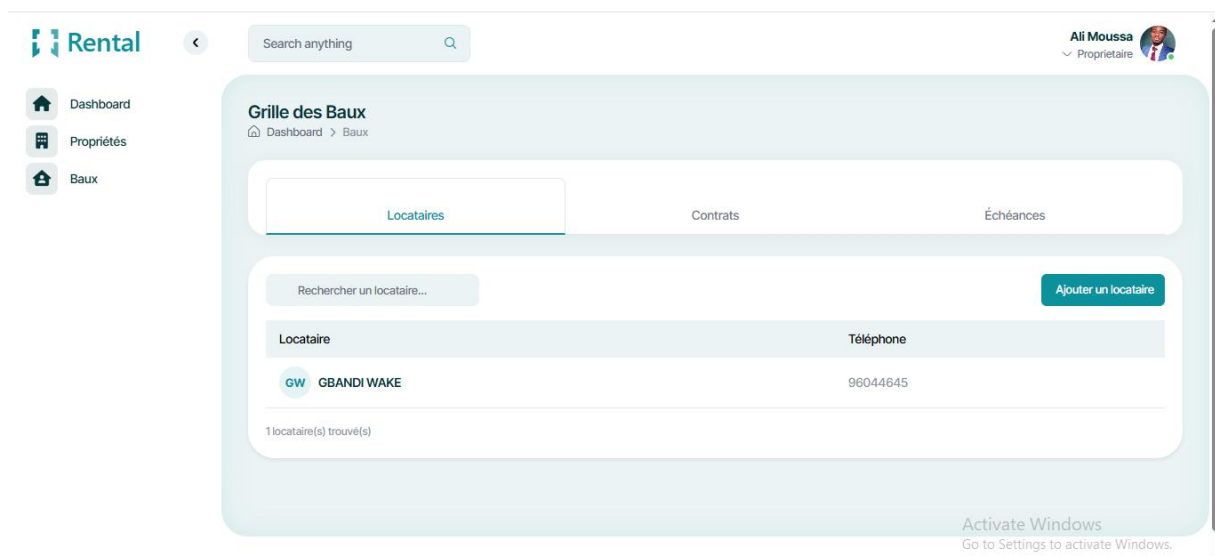


Figure 4.3 : page web de gestion des baux

Grille des Baux

Dashboard > Baux

Locataires

Contrats

Échéances

Rechercher un contrat...

Nouveau contrat

Locataire	Unité	Date début	Date fin	Loyer mensuel	Statut	Actions
GBANDI WAKE 96044645	APT-01	01/03/2026	—	40,000 FCFA	Actif	Résilier
GBANDI WAKE 96044645	BTQ-01	02/01/2026	—	30,000 FCFA	Actif	Résilier

Activate Windows

Figure 4.3 : page web de gestion des contrats

Nouveau contrat de bail

Locataire *

GBANDI WAKE (96044645)

Unité Logement *

BOUTIQUE - BTQ-01 - IMMEUBLE-Résidence de

Loyer mensuel convenu (FCFA) *

30000

FCFA

Jour d'échéance mensuel *

Chaque 3 du mois

Date de début / Prise d'effet *

02/01/2026

Date de fin (optionnelle)

dd/mm/yyyy

Clauses ou conditions particulières

N/A

Annuler

Créer le contrat

Figure 4.3 : formulaire d'ajout d'un contrat

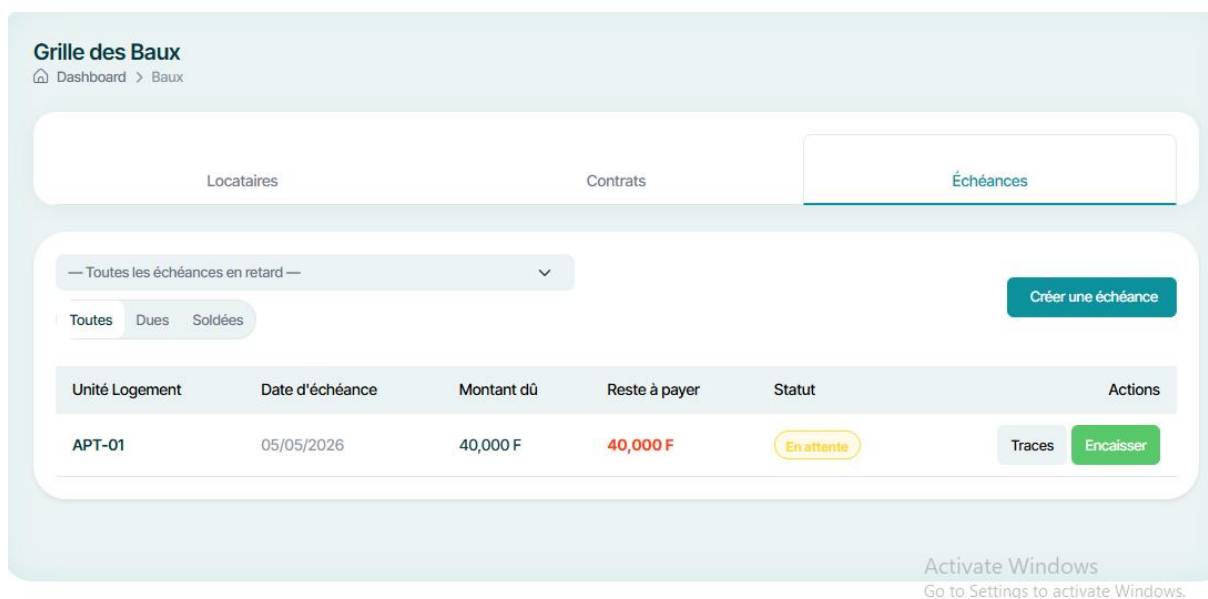


Figure 4.3 : pages web de gestion des échéances

(cette partie ci-dessous est gérée par la scheduler mais pour une raison de démonstration nous l'avons inséré)

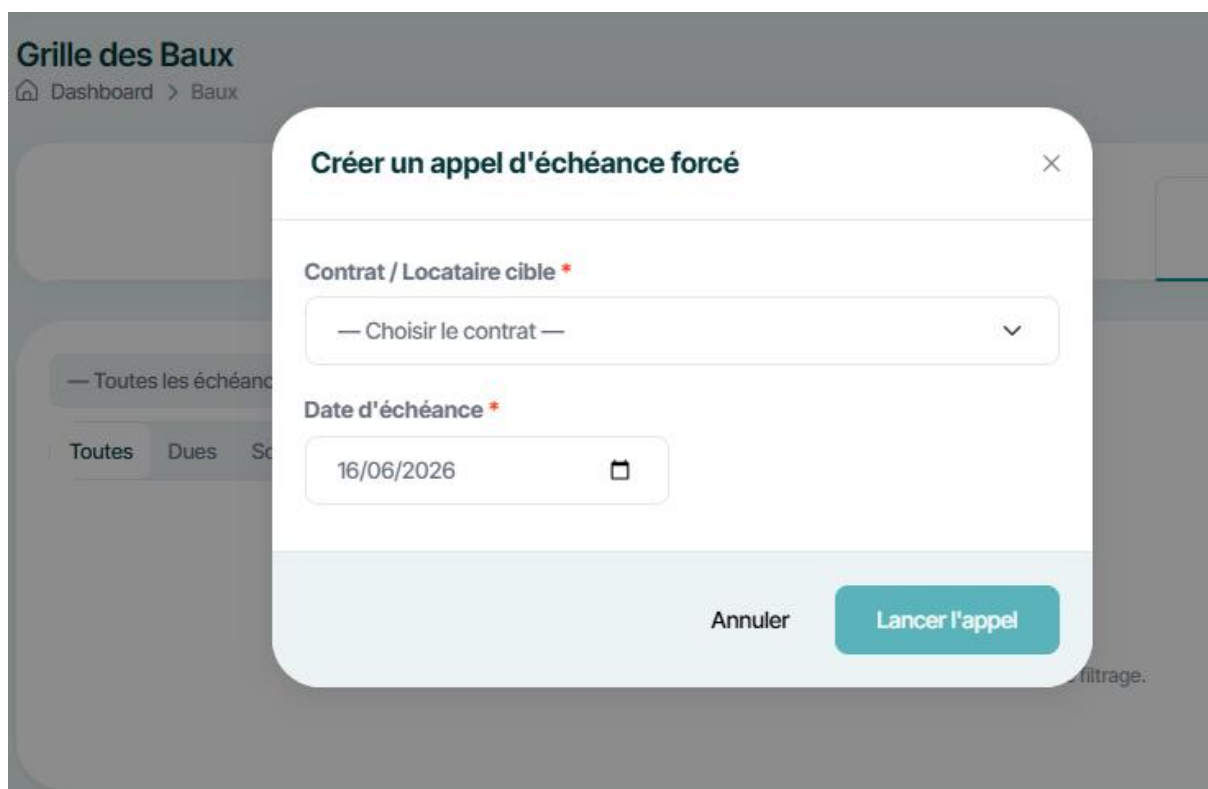


Figure 4.3 : formulaire d'ajout d'une échéance

Grille des Baux
Dashboard > Baux

Rechercher un locataire

Locataire	Téléphone
GW GBANDI WAKE	96044645

1 locataire(s) trouvé(s)

Nouveau locataire

Nom complet *

Ex: Koffi ALI

Téléphone *

+228 90 00 00 00

Ce numéro servira d'identifiant unique de connexion pour le locataire.

Mot de passe

Annuler Enregistrer

Figure 4.3 : formulaire d'ajout d'un locataire

Bonjour
GBANDI WAKE

LOYER MENSUEL TOTAL: 70000 F (2 contrats actifs)

Mes contrats de bail

- APT-01 APPARTEMENT** (ACTIF)
Échéance le 5 de chaque mois: 40000 F/mois
- BTQ-01 BOUTIQUE** (ACTIF)
Échéance le 3 de chaque mois: 30000 F/mois

Détail du contrat

APT-01 APPARTEMENT (ACTIF)

INFORMATIONS

Loyer mensuel	40000 F
Jour d'échéance	5 ^e du mois
Début du bail	1 mars 2026
Locataire	GBANDI WAKE
Téléphone	96 04 46 45
Conditions	Paielements à temps des échéances

Échéances

Aucune échéance générée pour ce contrat.

Figure 4.3 : Dashboard mobile & détail d'un contrat

4.3.2. Captures de codes sources

```
68 // Locataires
69
70 @PostMapping("add_locataire")
71 @PreAuthorize("hasRole('PROPRIETAIRE')")
72 public ResponseEntity<UtilisateurResponse> addLocataire(@RequestBody UtilisateurRequest request) {
73     return ResponseEntity.ok(contratBailService.addLocataire(request));
74 }
75
76 @GetMapping("mes_locataires")
77 @PreAuthorize("hasRole('PROPRIETAIRE')")
78 public ResponseEntity<List<UtilisateurResponse>> mesLocataires() {
79     Utilisateur userDb = ur.findByTelephone(authUtils.getTelephone()).orElseThrow();
80
81     List<UtilisateurResponse> response = userDb.getLocatairesCreer()
82         .stream()
83         .map(u -> new UtilisateurResponse(
84             u.getUsername(),
85             u.getProfil(),
86             u.getRole(),
87             u.getNomComple(),
88             u.getAvatar(),
89             u.getUuid())
90         )
91         .toList();
92     return ResponseEntity.ok(response);
93 }
94
95 @GetMapping("unite_libre")
96 @PreAuthorize("hasRole('PROPRIETAIRE')")
97 public ResponseEntity<List<UniteResponse>> getUniteLibre(){
98     return ResponseEntity.ok(contratBailService.getUniteLibre());
99 }
100
101 // Contrats
102
103 @PostMapping("add")
104 @PreAuthorize("hasRole('PROPRIETAIRE')")
105 public ResponseEntity<BailResponse> creerContrat(@RequestBody BailRequest request) {
106     return ResponseEntity.ok(contratBailService.addContrat(request));
107 }
108
109 @GetMapping("mes_contrats")
110 @PreAuthorize("hasRole('PROPRIETAIRE') or hasRole('LOCATAIRE')")
111 public ResponseEntity<List<BailResponse>> mesContrats() {
112     UUID locataireUuid = authUtils.getUtilisateurConnecte().getUuid();
113     return ResponseEntity.ok(contratBailService.getContratsByLocataire(locataireUuid));
114 }
115
116 @GetMapping("proprio_contrats")
117 @PreAuthorize("hasRole('PROPRIETAIRE') or hasRole('LOCATAIRE')")
118 public ResponseEntity<List<BailResponse>> proprioContrats() {
119     UUID id = authUtils.getUtilisateurConnecte().getUuid();
120     return ResponseEntity.ok(contratBailService.getContratsByProprietaire(id));
121 }
122
123 @PatchMapping("resilier/{contratUuid}")
124 @PreAuthorize("hasRole('PROPRIETAIRE')")
125 public ResponseEntity<BailResponse> resilier(@PathVariable UUID contratUuid) {
126     return ResponseEntity.ok(contratBailService.resilierContrat(contratUuid));
127 }
128
129 // Échéances
130
131 @GetMapping("echeances/{contratUuid}")
132 @PreAuthorize("hasRole('PROPRIETAIRE') or hasRole('LOCATAIRE')")
133 public ResponseEntity<List<EcheanceResponse>> getEcheances(@PathVariable UUID contratUuid) {
134     return ResponseEntity.ok(echeanceService.getEcheancesByContrat(contratUuid));
135 }
```

Figure 4.3 : Code source du controleur de bail


```

23 constructor(private http: HttpClient) {} no usages xavier
24
25 // Locataires
26
27 /**
28  * Crée un nouveau locataire.
29  * POST /api/bail/add_locataire
30  * Rôle requis : PROPRIETAIRE
31  */
32 addLocataire(request: UserRequest): Observable<UserInfo> { Show usages xavier
33     return this.http.post<UserInfo>({
34         url: `${this.baseUrl}/add_locataire`,
35         request
36     });
37 }
38
39 /**
40  * Récupère la liste des locataires créés par le propriétaire connecté.
41  * GET /api/bail/mes_locataires
42  * Rôle requis : PROPRIETAIRE
43  */
44 mesLocataires(): Observable<UserInfo[]> { Show usages xavier
45     return this.http.get<UserInfo[]>({
46         url: `${this.baseUrl}/mes_locataires`

```

Figure 4.3 : Code source du fichier service angular

```

41 // Paiement Mobile Money (Fedapay)
42 /**
43  * Initie un paiement Mobile Money (Flopz/TMoney).
44  * POST /api/bail/initier-paiement-mobile
45  */
46
47 initierPaiementMobile(request: InitPaiementMobileRequest): Observable<InitPaiementMobileResponse> { Show usages xavier
48     return this.http.post<InitPaiementMobileResponse>(`${this.baseUrl}/initier-paiement-mobile`, request);
49 }
50
51 // Notifications
52 /**
53  * Notifications non lues du locataire connecté.
54  * GET /api/bail/notifications/non-lues
55  */
56
57 getNotificationsNonLues(): Observable<NotificationResponse[]> { Show usages xavier
58     return this.http.get<NotificationResponse[]>(`${this.baseUrl}/notifications/non-lues`);
59 }
60
61 /**
62  * Marquer une notification comme lue.
63  * PATCH /api/bail/notifications/{notifUuid}/lire
64  */
65
66 marquerNotificationLue(notifUuid: string): Observable<NotificationResponse> { Show usages xavier
67     return this.http.patch<NotificationResponse>(`${this.baseUrl}/notifications/${notifUuid}/lire`, {});
68 }
69
70 }

```

Figure 4.3 : Code source du fichier service Ionic

Chapitre 5 : GUIDE DE DEPLOIEMENT ET D'EXPLOITATION

5.1. Configurations matérielles et logicielles

Pour garantir un déploiement stable, une exécution performante et une disponibilité continue de la plateforme web et mobile, les environnements d'hébergement (serveur) et d'exécution (client) doivent répondre à des exigences techniques minimales. Les spécifications requises sont réparties comme suit :

1. Spécifications requises côté Serveur (Hébergement)

Pour supporter le backend Spring Boot, le stockage PostgreSQL et assurer le traitement des requêtes en temps réel, l'infrastructure serveur (qu'elle soit locale ou sur une plateforme Cloud comme Render) doit disposer au minimum de :

- Mémoire Vive (RAM) : 1 Go de RAM minimum (2 Go recommandés pour absorber les pics de connexion et l'exécution fluide des tâches planifiées d'envoi de notifications push).
- Processeur : 1 vCPU (processeur virtuel) ou équivalent.
- Stockage : 5 Go d'espace disque disponible (technologie SSD de préférence) pour l'installation du système, des binaires et la croissance de la base de données relationnelle.
- Environnement Java : Java Runtime Environment (JRE) ou JDK version 17 ou supérieure, indispensable pour exécuter l'artéfact compilé de notre application Spring Boot.
- Environnement Node.js : Node.js version 18.x (LTS) ou supérieure, nécessaire sur la machine de build pour compiler l'application web Angular et générer les livrables mobiles Ionic.
- Système d'exploitation : Linux (Ubuntu Server 20.04 LTS ou supérieur recommandé), Windows Server ou toute plateforme d'exécution de conteneurs (Docker).

2. Spécifications requises côté Client (Utilisateurs)

L'accès aux interfaces applicatives a été optimisé pour s'adapter aux terminaux du contexte togolais sans exiger de configurations haut de gamme :

- Interface Web (Espace Propriétaire - Angular) :
 - ✓ Logiciel : Un navigateur web moderne supportant les standards HTML5, CSS3 et JavaScript ES6 (Google Chrome, Mozilla Firefox, Microsoft Edge ou Safari dans leurs versions récentes).
 - ✓ Matériel : Tout ordinateur (PC/Mac) ou tablette doté d'une connexion Internet stable.

- Interface Mobile (Espace Locataire - Ionic/Angular) :
 - ✓ Système d'exploitation : Android version 8.0 (Oreo) ou supérieure (et iOS 13 ou supérieur pour l'écosystème Apple), conformément à nos contraintes et exigences non fonctionnelles.
 - ✓ Connectivité : Une connexion Internet (3G/4G ou Wi-Fi) active, requise pour interroger l'API REST en temps réel, recevoir les notifications d'échéances et initialiser la passerelle de paiement Mobile Money Fedapay.

5.2. Guide de déploiement

Le déploiement de notre plateforme s'articule autour de trois composants principaux : l'API Backend, l'interface Web d'administration et l'application Mobile destinée aux terminaux Android. Les étapes suivantes décrivent le processus de compilation (*build*) et de mise en production de chaque brique du système :

1. Compilation et Déploiement du Backend (Spring Boot) avec Docker

Il est bon de savoir ce qu'est Docker ?

Docker est une technologie de conteneurisation utilisée pour packager l'application backend Spring Boot et ses dépendances (le JRE Java 17) sous forme d'une image isolée, légère et portable, facilitant son déploiement identique en local et en production.

Ainsi pour conteneuriser et déployer l'API Backend Spring Boot, la démarche suivante a été mise en œuvre :

Étape 1 : Rédaction du fichier de configuration (Dockerfile)

À la racine du projet backend, un fichier nommé Dockerfile a été créé pour définir les étapes de construction de l'image Docker de l'application :

```

1 # ---- Étape 1 : Construction de l'application ----
2 FROM maven:3.9.6-eclipse-temurin-17 AS build
3 WORKDIR /app
4
5 # Copie des fichiers de configuration Maven
6 COPY pom.xml .
7 # Téléchargement des dépendances (mise en cache pour accélérer les futurs builds)
8 RUN mvn dependency:go-offline -B
9
10 # Copie du code source et compilation
11 COPY src ./src
12 RUN mvn clean package -DskipTests
13
14 # ---- Étape 2 : Exécution de l'application ----
15 FROM eclipse-temurin:17-jre-jammy
16 WORKDIR /app
17
18 # Récupération du fichier JAR compilé à l'étape précédente
19 COPY --from=build /app/target/*.jar app.jar
20
21 # Port exposé par Render (8080 par défaut pour Spring Boot)
22 EXPOSE 8080
23
24 # Commande de démarrage de l'application
25 ENTRYPOINT ["java", "-jar", "app.jar"]

```

Figure 5.2 : fichier docker

Étape 2 : Construction (*Build*) et Test de l'image en local

1. Compiler d'abord le projet avec Maven pour générer le fichier JAR :

```
USER@WAKE MINGW64 ~  
$ mvn clean package -DskipTests
```

2. Construire l'image Docker locale nommée rentalapi :

```
USER@WAKE MINGW64 ~  
$ docker build -t rentalapi:
```

3. Exécuter le conteneur en local pour vérifier son bon fonctionnement :

```
USER@WAKE MINGW64 ~  
$ docker run -p 8080:8080 rentalapi:
```

Étape 3 : Déploiement automatique sur Render via Docker

La plateforme Cloud Render gère nativement les **Dockerfile**. Lors du déploiement :

- Sur le tableau de bord de Render, dans les paramètres du *Web Service*, l'environnement d'exécution (Runtime) a été modifié de *Java* à *Docker*.
- À chaque mise à jour du code sur le dépôt GitHub, Render détecte automatiquement la présence du Dockerfile.
- Render télécharge le code, exécute le processus de build Docker à distance, configure les variables d'environnement (clés JWT, accès PostgreSQL, clés Fedapay) et déploie le conteneur sécurisé sur son infrastructure Cloud.

2. Compilation et Déploiement de l'Application Web (Angular)

L'interface d'administration destinée aux propriétaires a été déployée en tant que site statique sur la plateforme Cloud **Render**. Grâce à l'intégration continue, le processus de build est entièrement automatisé à chaque mise à jour du code source sur le Github.

Étape 1 (Configuration des variables d'environnement) : Dans le code source Angular, le fichier « `src/environments/environment.prod.ts` » a été configuré pour pointer dynamiquement vers l'URL de production de notre API backend Spring Boot hébergée sur Render (<https://rental-api-ubpk.onrender.com>).

Étape 2 (Configuration du service sur Render) :

- Sur le tableau de bord de Render, créer un nouveau service de type Static Site.

- Connecter le dépôt GitHub sécurisé contenant le projet frontend Angular.

Étape 3 (Paramétrage des commandes de Build) : Dans les options de déploiement de Render, renseigner les paramètres suivants pour automatiser la génération des fichiers statiques :

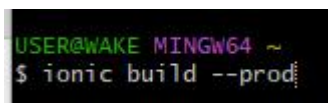
- Build Command : `npm install && npm run build -- --configuration production`. Cette commande indique à Render d'installer les dépendances Node.js puis de compiler l'application en mode production.
- Publish Directory : `dist/rental` («rental» représente le dossier généré par notre compilateur Angular contenant les fichiers `index.html`, CSS et JavaScript minifiés).

Étape 4 (Mise en production) : À la fin de la build automatique, Render génère une URL publique sécurisée (HTTPS) permettant au propriétaire d'accéder à son tableau de bord depuis n'importe quel navigateur moderne (<https://rental-web-pm3e.onrender.com>).

3. Génération du package de l'Application Mobile (Ionic / Android)

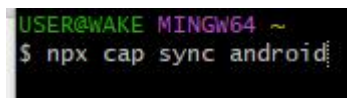
Pour distribuer l'application aux locataires, il est nécessaire de compiler le code web Ionic/Angular et de générer un fichier exécutable Android (.apk).

Étape 1 (Build du code web Ionic) : À la racine du projet mobile, lancer la compilation des fichiers sources TypeScript :



```
USER@WAKE MINGW64 ~
$ ionic build --prod
```

Étape 2 (Synchronisation avec Capacitor) : Exécuter la commande suivante pour copier les fichiers compilés dans le dossier natif Android configuré via le pont Capacitor :



```
USER@WAKE MINGW64 ~
$ npx cap sync android
```

Étape 3 (Génération de l'APK dans Android Studio) :

- Ouvrir le dossier `android/` du projet mobile à l'aide du logiciel Android Studio.
- Attendre la fin de la synchronisation du moteur de build Gradle.
- Dans le menu supérieur, aller sur `Build > Build Bundle(s) / APK(s) > Build APK(s)`.

Étape 4 (Livrable final) : Une fois la compilation terminée, Android Studio génère le fichier `app-debug.apk` ou `app-release.apk` (situé dans `android/app/build/outputs/apk/`). Ce fichier est ensuite transféré sur le smartphone Android de test pour valider les flux de paiement Mobile Money et le système de notifications push en situation réelle.

5.3. Guide d'exploitation

Le guide d'exploitation regroupe l'ensemble des procédures nécessaires pour assurer la maintenance préventive, la surveillance et la continuité du service de la plateforme après sa mise en production. Il fournit à l'administrateur les clés pour garantir la sécurité des données et la fiabilité des flux financiers.

5.3.1. Procédure de sauvegarde de la base de données (Supabase / PostgreSQL)

La persistance de l'ensemble des données immobilières, des contrats et des historiques de paiement est entièrement déléguée à Supabase, qui propulse notre base de données PostgreSQL en production. Afin de garantir la résilience face aux pannes, la stratégie de sauvegarde s'articule comme suit :

- Sauvegardes automatisées quotidiennes : Supabase réalise automatiquement des clichés (*snapshots*) complets de la base de données toutes les 24 heures. Ces sauvegardes incluent la structure des tables, les contraintes d'intégrité et les enregistrements. Elles sont conservées de manière sécurisée et peuvent être restaurées en un clic depuis le tableau de bord de Supabase.
- Sauvegarde logique manuelle (Export) : En cas de maintenance majeure, l'administrateur peut exporter instantanément le schéma et les données en local en utilisant l'utilitaire standard de PostgreSQL via la chaîne de connexion sécurisée fournie par Supabase :

```
USER@WAKE MINGW64 ~  
$ pg_dump "postgres://postgres:[votre_mot_de_passe]@pooler.supabase.com:6543/postgres" -F c -b -v -f sauvegarde_immo_$(date +%F).dump
```

5.3.2. Surveillance des logs et état du système

La surveillance de la plateforme est répartie entre les deux couches d'infrastructure Cloud pour faciliter le diagnostic en cas d'incident :

- Logs applicatifs (Render) : L'onglet «Logs» du conteneur Docker de Render permet de suivre l'activité en temps réel du backend Spring Boot. L'administrateur doit y surveiller les exceptions de sécurité (jetons JWT altérés ou expirés) et les codes d'erreur HTTP de la série `5xx`.
- Logs de persistance (Supabase) : Le tableau de bord de Supabase fournit à l'administrateur une interface de surveillance dédiée à la base de données. Elle permet de suivre l'état de santé du serveur, le nombre de connexions actives au pool, et d'analyser les logs des requêtes SQL pour identifier d'éventuelles lenteurs.

5.3.3. Gestion des tokens FedaPay : Passage de Sandbox à Production

Le système utilise la passerelle togolaise FedaPay pour orchestrer les transactions Mobile Money (Flooz et T-Money). Pour passer de la phase de test à l'exploitation réelle par les utilisateurs, la procédure de bascule des clés d'API doit être respectée :

Table 3 : Règle d'exploitation fedapay

Paramètres	Phase de Test (Sandbox)	Phase d'Exploitation (Production)
Role	Simuler des paiements fictifs sans flux d'argent réel.	Traiter les vrais règlements des locataires locaux.
Clé Secrète API	`sk_sandbox_...`	`sk_live_...` (Générée après validation du compte marchand)
Validation Webhook	Clé secrète HMAC de test pour authentifier les notifications.	Nouvelle clé secrète HMAC de production fournie par FedaPay.

Règle d'exploitation critique : Le changement de mode s'effectue sans aucune modification du code source de l'application. Il suffit à l'administrateur de mettre à jour les variables d'environnement (`FEDAPAY\_SECRET\_KEY` et `FEDAPAY\_WEBHOOK\_SECRET`) dans l'onglet «Environment» du service Render, puis de redémarrer le conteneur Docker pour que la plateforme bascule instantanément sur le réseau bancaire réel.

Chapitre 6 : GUIDE D'UTILISATION

Ce chapitre présente les manuels opératoires de la plateforme à destination de chacun des deux profils d'utilisateurs identifiés lors de la conception : le Propriétaire, qui administre son patrimoine immobilier depuis l'interface web, et le Locataire, qui consulte ses échéances et règle son loyer depuis l'application mobile. Chaque procédure est illustrée par les captures d'écran correspondantes (Figures 6.1 à 6.x), insérées au fil des étapes décrites.

6.1 Manuel opératoire du Propriétaire (interface web)

L'interface web, développée avec Angular et accessible à l'adresse <https://rental-web-pm3e.onrender.com>, constitue le poste de pilotage du propriétaire. Elle lui permet de structurer son patrimoine (biens et unités), de gérer ses locataires et ses contrats, puis de suivre l'exécution financière de ses baux.

6.1.1. Connexion à la plateforme

Étape 1 (Authentification) : Sur la page de connexion, le propriétaire renseigne son numéro de Téléphone et son Code d'accès, puis valide via le bouton Login.

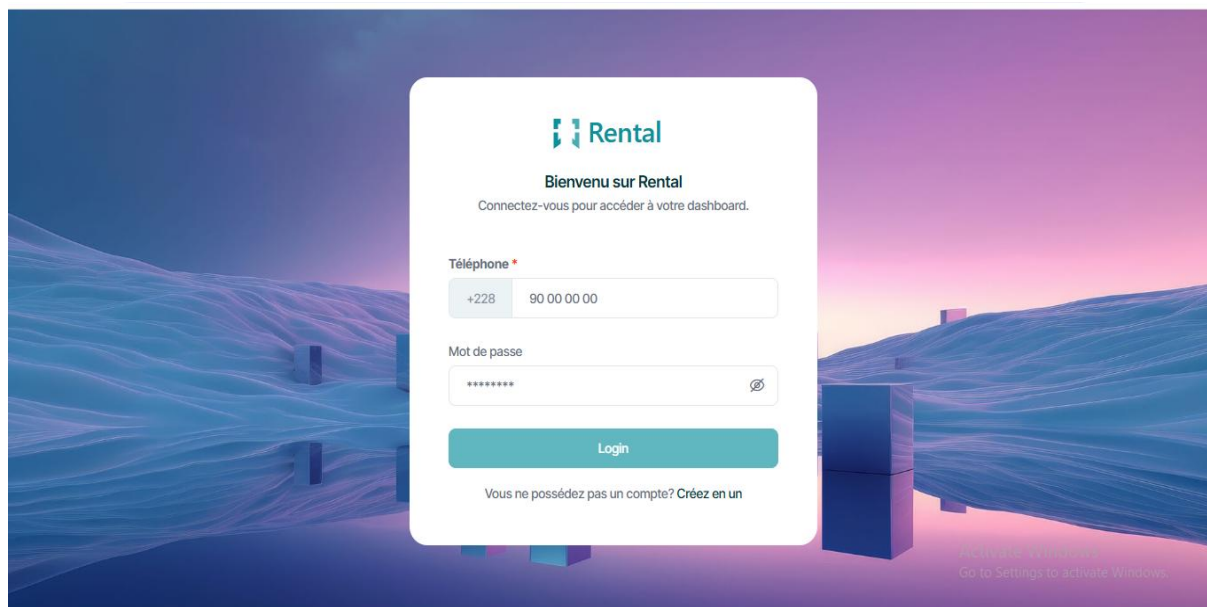


Figure 6.1 : Ecran de d'authentification

Étape 2 (Redirection) : Après vérification des identifiants par le backend (génération d'un jeton JWT), l'utilisateur est automatiquement redirigé vers son tableau de bord, dont la nature dépend de son profil d'activité : *Location Classique* ou *Location Séjour (Airbnb)*.

6.1.2. Ajout d'un bien

Étape 1 : Depuis le menu latéral, accéder à la rubrique Propriétés.



Figure 6.2 : Menu du propriétaire

Étape 2 : Cliquer sur le bouton Ajouter un Bien.

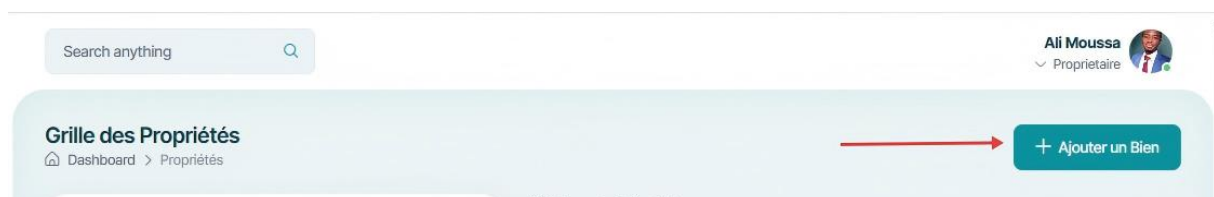


Figure 6.3 : Bouton d'ajout d'un bien

Étape 3 : Renseigner le libellé du bien, son type (*Immeuble* ou *Cour Commune*), la ville et le quartier, puis valider par Enregistrer.

Ajouter un nouveau bien

Libellé du bien
Ex: Résidence Amina

Type de bien
Immeuble

Ville
Ex: Lomé

Quartier
Ex: Adidogomé

Annuler Enregistrer

Figure 6.4 : Formulaire d'ajout d'un bien

6.1.3. Création d'une unité

Étape 1 : Depuis la fiche d'un bien, cliquer sur Unités liées pour accéder à la liste des logements qui le composent.

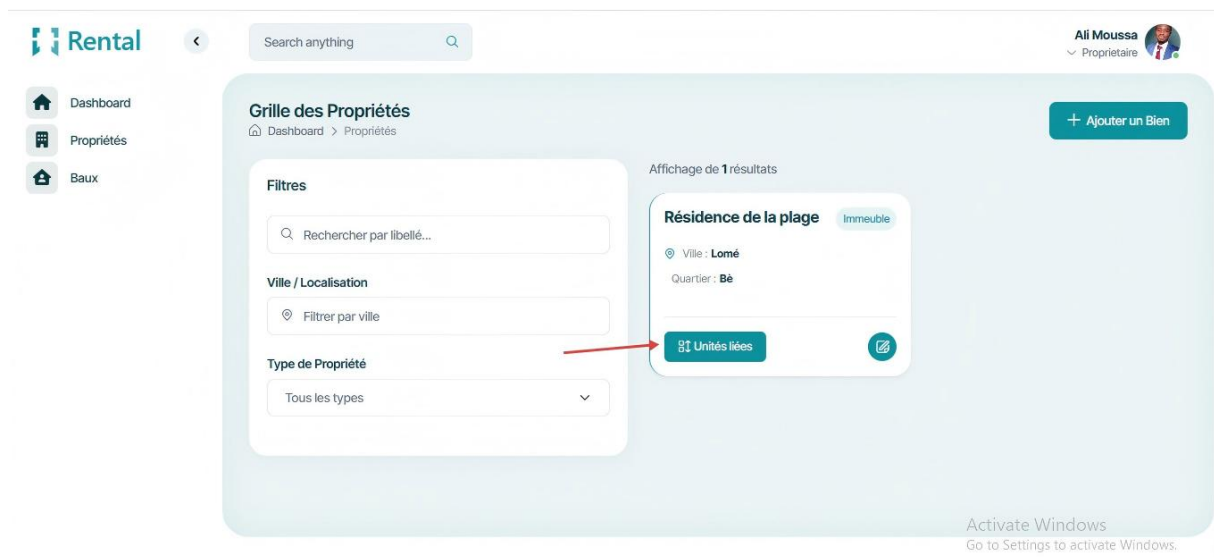


Figure 6.5 : Bouton d'accès aux unités

Étape 2 : Cliquer sur Ajouter une Unité.

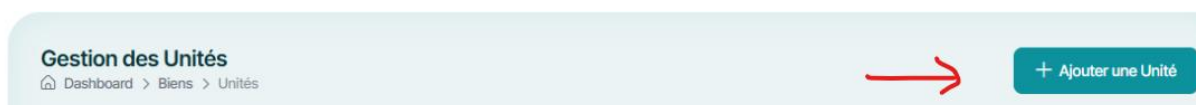


Figure 6.6 : Bouton d'ajout d'une unité de logement

Étape 3 : Renseigner le code de l'unité, son type (Appartement, Villa, Studio, Boutique), le loyer de référence ainsi que son statut initial (*Libre*), puis Enregistrer. L'unité apparaît désormais disponible pour l'attribution d'un futur contrat.

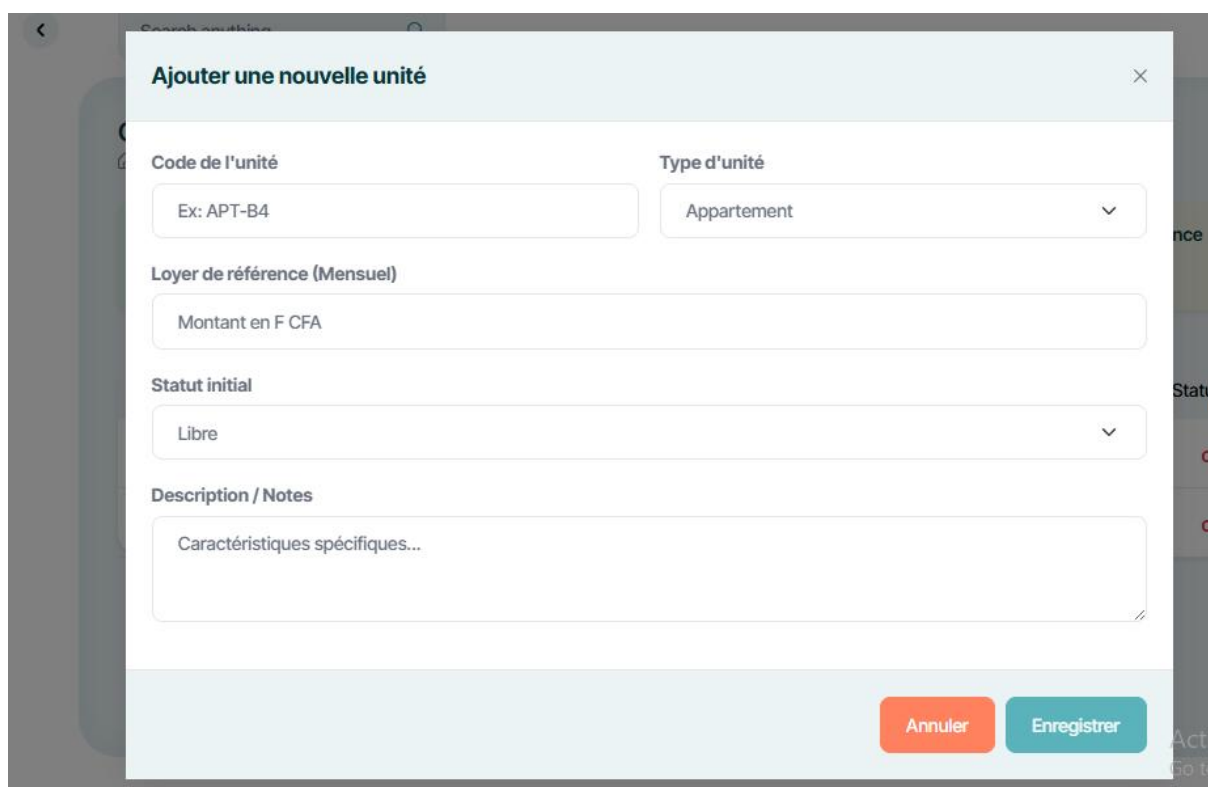


Figure 6.7 : Formulaire d'ajout d'une unité

6.1.4. Création d'un locataire

Étape 1 : Accéder à la rubrique Baux, puis à l'onglet Locataires.

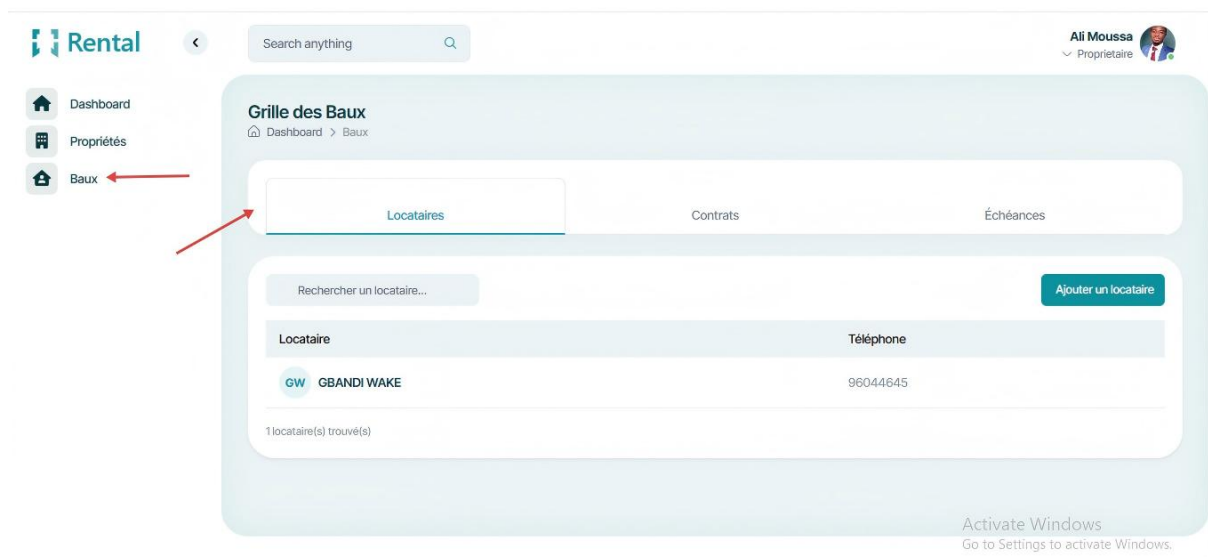


Figure 6.8 : Bouton d'accès à la rubrique des baux

Étape 2 : Cliquer sur Ajouter un locataire.



Figure 6.9 : Bouton d'ajout d'un locataire

Étape 3 : Saisir le nom complet, le numéro de téléphone (qui servira d'identifiant de connexion) et un code d'accès provisoire, puis valider. Le compte du locataire est créé et rattaché au propriétaire connecté.

The screenshot shows a modal form titled 'Nouveau locataire'. It has three input fields: 'Nom complet' with a red asterisk and an example 'Ex: Koffi ALI', 'Téléphone' with a red asterisk and an example '+228 90 00 00 00', and 'Mot de passe' with a red asterisk and a masked input. Below the 'Téléphone' field, there is a note: 'Ce numéro servira d'identifiant unique de connexion pour le locataire.' At the bottom, there are two buttons: 'Annuler' and 'Enregistrer'.

Figure 6.10 : Formulaire d'enregistrement d'un locataire

6.1.5. Création d'un contrat de bail

Étape 1 : Depuis l'onglet Contrats, cliquer sur Nouveau contrat.

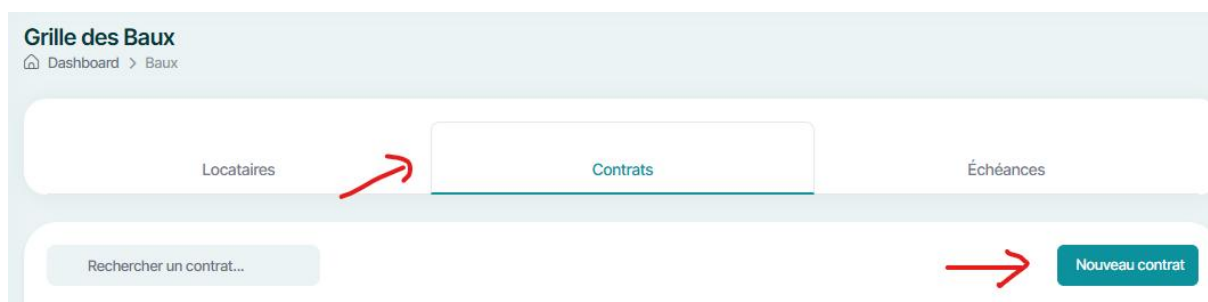


Figure 6.11 : Onglet contrats

Étape 2 : Sélectionner le locataire concerné, puis l'unité libre à lui attribuer (le loyer mensuel se pré-remplit automatiquement à partir du loyer de référence de l'unité), ensuite Préciser le jour d'échéance mensuel, la date de prise d'effet, une éventuelle date de fin, ainsi que les clauses particulières, puis valider par Créer le contrat.

The screenshot shows a form titled 'Nouveau contrat de bail' with a close button (X) in the top right corner. The form contains several fields: 'Locataire *' with a dropdown menu showing 'GBANDI WAKE (96044645)'; 'Unité Logement *' with a dropdown menu showing 'BOUTIQUE - BTQ-01 - IMMEUBLE-Résidence de'; 'Loyer mensuel convenu (FCFA) *' with a text input '30000' and a dropdown menu 'FCFA'; 'Jour d'échéance mensuel *' with a dropdown menu showing 'Chaque 3 du mois'; 'Date de début / Prise d'effet *' with a date input '02/01/2026' and a calendar icon; 'Date de fin (optionnelle)' with a date input 'dd/mm/yyyy' and a calendar icon; and 'Clauses ou conditions particulières' with a text area containing 'N/A'. At the bottom right of the form are two buttons: 'Annuler' and 'Créer le contrat'.

Figure 6.12 : Formulaire d'enregistrement d'un contrat

Étape 4 (Effets automatiques) : La création du contrat déclenche le passage du statut de l'unité à *Occupée* ainsi que l'envoi automatique d'une notification de bienvenue au locataire, l'informant du montant de son loyer et de la date de sa première échéance.

6.1.6. Suivi des échéances et encaissement

Étape 1 : Depuis l'onglet Échéances, le propriétaire visualise l'ensemble des appels de loyer, avec un filtrage par statut (*Toutes, Dues, Soldées*).

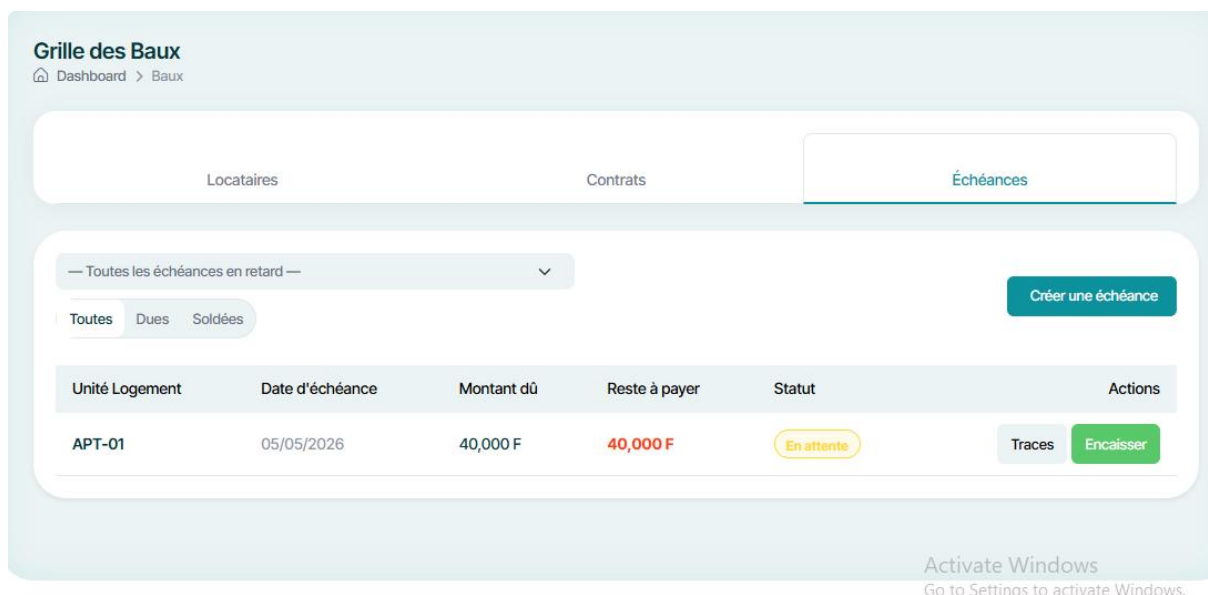


Figure 6.13 : Onglet des Echéances

Étape 2 (Encaissement en espèces) : Pour une échéance non soldée, cliquer sur Encaisser, saisir le montant versé par le locataire, puis Valider. Le système met à jour automatiquement le montant restant dû ainsi que le statut de l'échéance, et notifie le locataire de la confirmation de son paiement.



Figure 6.14 : Bouton d'encaissement de loyer

Étape 3 (Historique) : Le bouton Traces permet de consulter l'historique détaillé des versements effectués sur une échéance donnée.



Figure 6.15 : Bouton d'accès aux historiques des encaissements

6.1.7. Résiliation d'un contrat

Étape 1 : Sur la ligne du contrat concerné, cliquer sur Résilier.

Locataire	Unité	Date début	Date fin	Loyer mensuel	Statut	Actions
GBANDI WAKE 96044645	APT-01	01/03/2026	—	40,000 FCFA	Actif	 Résilier

Figure 6.16 : Bouton de résiliation d'un contrat

6.2 Manuel opératoire du locataire (interface mobile)

L'application mobile, développée avec Ionic/Angular et packagée via Capacitor, offre au locataire un accès simplifié à son contrat, à ses échéances et au règlement de son loyer par Mobile Money, où qu'il se trouve.

6.2.1. Connexion à l'application

Étape 1 : Sur l'écran de connexion, le locataire saisit le numéro de téléphone et le code d'accès qui lui ont été communiqués par son propriétaire, puis appuie sur Se connecter.



Un contrôle d'accès (*Auth Guard*) vérifie que le compte connecté possède bien le rôle *Locataire* avant de donner accès au tableau de bord

Figure 6.17 : Ecran d'authentification du mobile

6.2.2. Consultation du contrat de bail

Étape 1 : Le tableau de bord présente la synthèse du loyer mensuel total ainsi que la liste des contrats actifs du locataire.

Étape 2 : En sélectionnant un contrat, le locataire accède au détail complet : montant du loyer, jour d'échéance, dates de début et de fin, conditions particulières, ainsi que l'historique chronologique de ses échéances avec leur statut respectif (*Payé*, *Partiel*, *En retard*, *En attente*).



Figure 6.18 : Dashboard locataire



Figure 6.19 : Détail d'un contrat

6.2.3. Paiement d'une échéance via Mobile Money

Étape 1 : Depuis le détail d'une échéance non soldée, le locataire appuie sur le bouton Payer par Mobile Money.

Étape 2 : L'application initie une transaction auprès de la passerelle Fedapay et redirige le locataire vers un écran récapitulatif (montant à régler, unité concernée).

Étape 3 : En appuyant sur Ouvrir la page de paiement, le locataire est dirigé vers la page sécurisée Fedapay, où il saisit son numéro Flooz ou T-Money et confirme l'opération sur son téléphone.

Étape 4 (Traitement automatique) : Une fois le paiement validé par l'opérateur Mobile Money, Fedapay notifie le backend via webhook signé, qui solde automatiquement l'échéance et met à jour son statut sans aucune intervention manuelle.



Figure 6.20 : Détail d'une échéance

6.2.4. Réception de la notification de confirmation

Étape 1 : Le locataire reçoit, dans l'onglet Notifications, un message de confirmation précisant le montant réglé et la référence de la transaction. Un badge de compteur, visible sur l'icône de notification de la barre de navigation, signale le nombre de messages non lus ; un appui sur la notification la marque comme lue

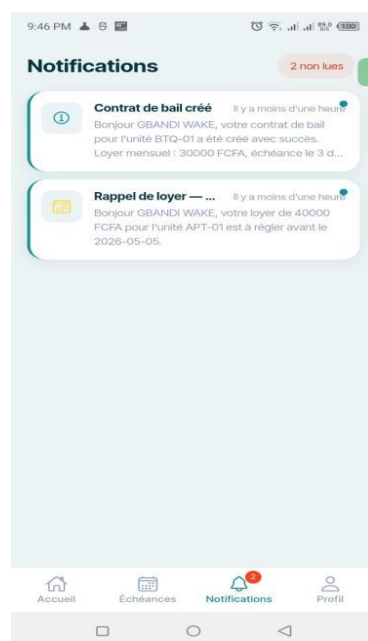


Figure 6.21 : Notification

CONCLUSION GENERALE

Bilan : Ce projet a permis de concevoir et de déployer une solution de gestion locative multi-cible, répondant aux réalités du marché togolais. La solution fait coexister avec succès l'administration à long terme (Profil A - bail classique) et à courte durée (Profil B - Airbnb) au sein d'une même architecture logicielle. L'intégration complète de la passerelle FedaPay sécurise les transactions par Mobile Money (Flooz, T-Money), tandis qu'un système de notifications push automatise le suivi des échéances en temps réel.

Apports : Sur le plan technique, ce travail a consolidé ma maîtrise de l'écosystème Java/Spring Boot en passant par la conteneurisation avec Docker, de la modélisation UML 2.x structurée jusqu'au développement d'API REST sécurisées par jetons JWT. Allant du recueil; des besoins à la gestion des environnements de production cloud sur Render et Supabase. Cette expérience a éclairé ma compréhension des architectures logicielles complexe.

Perspectives : Plusieurs évolutions majeures sont envisagées pour enrichir la plateforme. À court terme, l'implémentation d'un module de génération automatique de quittances de loyer au format PDF. À plus long terme, le déploiement d'une version iOS native via Ionic et l'intégration d'un modèle d'intelligence artificielle dédié à la prévision des risques d'impayés permettraient de transformer ce projet d'études en une solution commerciale.

BIBLIOGRAPHIE ET WEBOGRAPHIE

Ouvrages et ressources techniques

WALLS, Craig. Spring in Action, 6^e édition, Manning Publications, 2022.

FREEMAN, Adam et MINUTILLO, Stephen. Pro Angular, 4^e édition, Apress, 2021.

ROFF, Matt. Pro JPA 2 in Java EE 8, Apress, 2018.

ROUMANES, Jean-Baptiste. UML 2 : Modélisation des systèmes d'information, Eyrolles, 2020.

Webographie

Spring Boot Reference Documentation, <https://docs.spring.io/spring-boot/> (consulté en 2026).

Documentation officielle Angular, <https://angular.dev/> (consulté en 2026).

Documentation officielle Ionic Framework, <https://ionicframework.com/docs> (consulté en 2026).

FedaPay – Documentation API de paiement Mobile Money, <https://docs.fedapay.com/> (consulté en 2026).

JWT.io – Introduction aux JSON Web Tokens, <https://jwt.io/introduction> (consulté en 2026).

PostgreSQL Documentation, <https://www.postgresql.org/docs/> (consulté en 2026).

OpenAPI Initiative – Spécification REST API, <https://www.openapis.org/> (consulté en 2026).